RDF 1.2 XML Syntax

W3C Working Draft 02 February 2026



▼ More details about this document

This version:

https://www.w3.org/TR/2026/WD-rdf12-xml-20260202/

Latest published version:

https://www.w3.org/TR/rdf12-xml/

Latest editor's draft:

https://w3c.github.io/rdf-xml/spec/

History:

https://www.w3.org/standards/history/rdf12-xml/

Commit history

Test suite:

https://w3c.github.io/rdf-tests/rdf/rdf12/rdf-xml/

Latest Recommendation:

https://www.w3.org/TR/rdf-syntax-grammar

Editor:

Gregg Kellogg (until 2025-09-06), in memoriam

Former editors:

Fabien Gandon

Guus Schreiber

Dave Beckett

Ora Lassilla

Ralph Swick

Author:

Dave Beckett

Feedback:

GitHub w3c/rdf-xml (pull requests, new issue, open issues)

public-rdf-star-wg@w3.org with subject line `[rdf12-xml] … message topic …` (archives)

Copyright © 2004-2026 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

Abstract

The Resource Description Framework (RDF) is a general-purpose language for representing information in the Web.

This document defines an XML syntax for RDF called RDF/XML in terms of Namespaces in XML, the XML Information Set [*XML-INFOSET*] and XML Base [*XMLBASE*].

RDF 1.2 XML Syntax introduces triple terms as a fourth kind of RDF term which can be used as the object of another triple, making it possible to make statements about other statements. Triple terms are typically not used explicitly as the annotation construction is generally preferred. RDF 1.2 XML Syntax also adds support for directional language-tagged strings.

Status of This Document

This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the W3C standards and drafts index.

This document is a part of the RDF 1.2 document suite. The document defines the RDF/XML syntax, a concrete syntax for RDF [*RDF12-CONCEPTS*].

There have been no substantive changes to this document since [*rdf-syntax-grammar*]. Minor editorial changes, if any, are detailed in B. Changes between RDF 1.1 and RDF 1.2.

This document was published by the RDF & SPARQL Working Group as a Working Draft using the Recommendation track.

Publication as a Working Draft does not imply endorsement by W3C and its Members.

This is a draft document and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to cite this document as other than a work in progress. Future updates to this upcoming Recommendation may incorporate new features.

This document was produced by a group operating under the W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent that the individual believes contains Essential Claim(s) must disclose the information in accordance with section 6 of the W3C Patent Policy.

This document is governed by the 18 August 2025 W3C Process Document.

§ Set of Documents

This document is one of eleven RDF 1.2 and twelve SPARQL 1.2 documents produced by the RDF & SPARQL Working Group.

▶ **List of documents**

Table of Contents

Abstract

Status of This Document
Set of Documents

1. Introduction

2. An XML Syntax for RDF
2.1 Introduction
2.2 Node Elements and Property Elements
2.3 Multiple Property Elements
2.4 Empty Property Elements
2.5 Property Attributes
2.6 Completing the Document: Document Element and XML Declaration
2.7 Languages: `xml:lang`
2.8 Base Direction: `its:dir`
2.9 XML Literals: `rdf:parseType="Literal"`

2.10 Typed Literals: `rdf:datatype`
2.11 Identifying Blank Nodes: `rdf:nodeID`
2.12 Omitting Blank Nodes: `rdf:parseType="Resource"`
2.13 Omitting Nodes: Property Attributes on an empty Property Element
2.14 Typed Node Elements
2.15 Abbreviating IRIs: `rdf:ID` and `xml:base`
2.16 Container Membership Property Elements: `rdf:li` and `rdf:_n`
2.17 Collections: `rdf:parseType="Collection"`
2.18 Reifying Statements: `rdf:ID`
2.19 Triple Terms: `rdf:parseType="Triple"`
2.20 Annotating Triples: `rdf:annotation` or `rdf:annotationNodeID`

3. Conformance
3.1 RDF/XML Internet Media Type, File Extension, and Macintosh File Type
3.2 Privacy Considerations
3.3 Security Considerations

4. Global Issues
4.1 The RDF Namespace and Vocabulary
4.2 Identifiers
4.3 Resolving IRIs
4.4 Constraints

5. Syntax Data Model
5.1 Events
5.1.1 Root Event
5.1.2 Element Event
5.1.3 End Element Event
5.1.4 Attribute Event
5.1.5 Text Event
5.1.6 IRI Event
5.1.7 Triple Term Event
5.1.8 Blank Node Identifier Event
5.1.9 Literal Without Datatype Event
5.1.10 Typed Literal Event
5.2 Information Set Mapping
5.3 Grammar Notation

5.3.1 Grammar General Notation
5.3.2 Grammar Event Matching Notation
5.3.3 Grammar Action Notation

6. RDF/XML Grammar
6.1 Grammar summary
6.2 Grammar Productions
6.2.1 Grammar start
6.2.2 Production coreSyntaxTerms
6.2.3 Production syntaxTerms
6.2.4 Production oldTerms
6.2.5 Production nodeElementIRIs
6.2.6 Production propertyElementURIs
6.2.7 Production propertyAttributeIRIs
6.2.8 Production doc
6.2.9 Production RDF
6.2.10 Production nodeElementList
6.2.11 Production nodeElement
6.2.12 Production ws
6.2.13 Production propertyEltList
6.2.14 Production propertyElt
6.2.15 Production resourcePropertyElt
6.2.16 Production literalPropertyElt
6.2.17 Production parseTypeLiteralPropertyElt
6.2.18 Production parseTypeResourcePropertyElt
6.2.19 Production parseTypeTriplePropertyElt
6.2.20 Production parseTypeCollectionPropertyElt
6.2.21 Production parseTypeOtherPropertyElt
6.2.22 Production emptyPropertyElt
6.2.23 Production idAttr
6.2.24 Production nodeIdAttr
6.2.25 Production aboutAttr
6.2.26 Production annotationAttr
6.2.27 Production annotationNodeIDAttr
6.2.28 Production propertyAttr
6.2.29 Production resourceAttr

6.2.30 Production datatypeAttr
6.2.31 Production parseLiteral
6.2.32 Production parseResource
6.2.33 Production parseTriple
6.2.34 Production parseCollection
6.2.35 Production parseOther
6.2.36 Production IRI
6.2.37 Production literal
6.2.38 Production rdf-id
6.3 Reification Rules
6.4 List Expansion Rules

7. Serializing an RDF Graph to RDF/XML

8. Using RDF/XML with SVG

A. Acknowledgments
A.1 Acknowledgments the original specification
A.2 Acknowledgments for RDF 1.1
A.3 Acknowledgments for RDF 1.2

B. Changes between RDF 1.1 and RDF 1.2

C. Syntax Schemas
C.1 RELAX NG Compact Schema

D. Index
D.1 Terms defined by this specification
D.2 Terms defined by reference

E. References
E.1 Normative references
E.2 Informative references

§ 1. Introduction

This section is non-normative.

This document defines the XML [*XML11*] syntax for RDF graphs.

This document revises the original RDF/XML grammar [*RDF-SYNTAX-GRAMMAR-19990222*] in terms of XML Information Set [*XML-INFOSET*] information items which moves away from the rather low-level details of XML, such as particular forms of empty elements. This allows the grammar to be more precisely recorded and the mapping from the XML syntax to the RDF Graph more clearly shown. The mapping to the RDF graph is done by emitting statements in the N-Triples [*RDF12-N-TRIPLES*] format.

- A document describing the basic concepts underlying RDF, as well as abstract syntax ("RDF Concepts and Abstract Syntax") [*RDF12-CONCEPTS*]
- A document describing the formal model-theoretic semantics of RDF ("RDF Semantics") [*RDF12-SEMANTICS*]
- Specifications of concrete syntaxes for RDF:
  - Turtle [*RDF12-TURTLE*] and TriG [*RDF12-TRIG*]
  - JSON-LD [*JSON-LD11*] (JSON based)
  - RDFa [*RDFA-PRIMER*] (for HTML embedding)
  - N-Triples [N-TRIPLES] and N-Quads [N-QUADS] (line-based exchange formats)
- A document describing RDF Schema [*RDF-SCHEMA*], which provides a data-modeling vocabulary for RDF data.

For a longer introduction to the RDF/XML syntax with a historical perspective, see "RDF: Understanding the Striped RDF/XML Syntax" [*STRIPEDRDF*].

§ 2. An XML Syntax for RDF

This section is non-normative.

This section introduces the RDF/XML syntax, describes how it encodes RDF graphs and explains this with examples. If there is any conflict between this informal description and the formal description of the syntax and grammar in sections 5. Syntax Data Model and 6. RDF/XML Grammar, the latter two sections take precedence.

§ 2.1 Introduction

The RDF Concepts and Abstract Syntax document [*RDF12-CONCEPTS*] defines the RDF data model and the RDF abstract syntax. Along with the RDF Semantics [*RDF12-SEMANTICS*] this provides an abstract syntax with a formal semantics for it. The RDF data model has nodes and labeled directed *arcs* that link pairs of nodes and this is represented as a set of RDF triples where each triple contains a *subject node*, *predicate* and *object node*. Nodes are IRIs, literals, or blank nodes. Blank nodes may be given a document-local identifier called a blank node identifier. Predicates are IRIs and can be interpreted as either a relationship between the two nodes or as defining an attribute value (object node) for some subject node.

In order to encode the graph in XML, the nodes and predicates have to be represented in XML terms — element names, attribute names, element contents and attribute values. RDF/XML uses XML QNames as defined in Namespaces in XML [*XML-NAMES*] to represent IRIs. All QNames have a ***namespace name*** which is an IRI and a short local name. In addition, QNames can either have a short prefix or be declared with the default namespace declaration and have no prefix (but still have a namespace name)

The IRI represented by a QName is determined by appending the local name part of the QName after the namespace name (IRI) part of the QName. This is used to shorten the IRI of all predicates and some nodes. IRIs identifying subject and object nodes can also be stored as XML attribute values. RDF literals which can only be object nodes, become either XML element text content or XML attribute values.

A graph can be considered a collection of paths of the form node, predicate arc, node, predicate arc, node, predicate arc, ... node which cover the entire graph. In RDF/XML these turn into sequences of elements inside elements which alternate between elements for nodes and predicate arcs. This has been called a series of node/arc stripes. The node at the start of the sequence turns into the outermost element, the next predicate arc turns into a child element, and so on. The stripes generally start at the top of an RDF/XML document and always begin with nodes.

Several RDF/XML examples are given in the following sections building up to complete RDF/XML documents. Example 7 is the first complete RDF/XML document.

§ 2.2 Node Elements and Property Elements

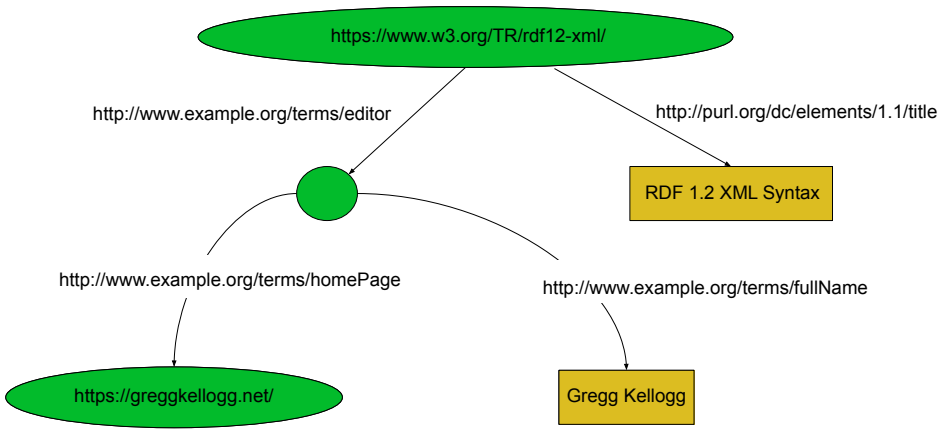


Figure 1 Graph for RDF/XML Example described in Example 4.

An RDF graph is given in Figure 1 where the nodes are represented as ovals which contain their IRIs where they have them, all the predicate arcs are labeled with IRIs, and string literal nodes have been written in rectangles.

If we follow one node, predicate arc ... , node path through the graph shown in Figure 2:

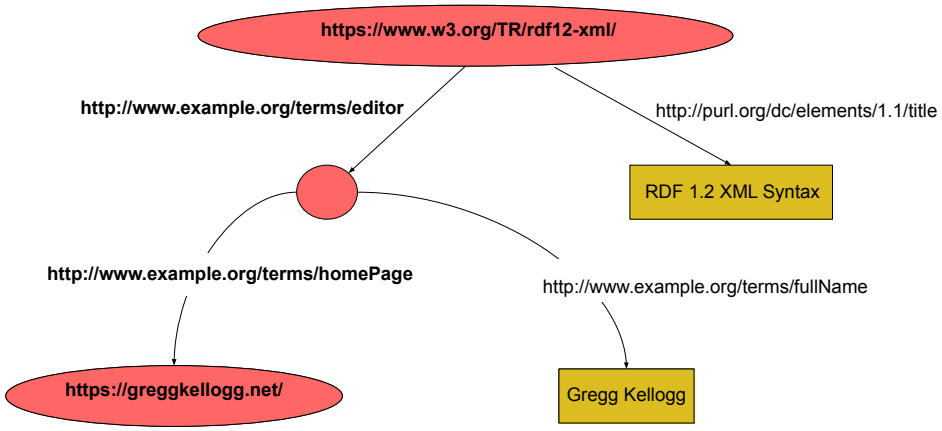


*Figure 2 One Path Through the Graph described in Example 4 highlighting the path from <https://www.w3.org/TR/rdf12-xml/> to <https://greggkellogg.net/>. Elements along the path are marked in **bold text**.*

The left-hand side of the Figure 2 graph corresponds to the node/predicate arc stripes:

1. Node with IRI `https://www.w3.org/TR/rdf12-xml/`
2. Predicate Arc labeled with IRI `http://example.org/terms/editor`
3. Node with no IRI
4. Predicate Arc labeled with IRI `http://example.org/terms/homePage`
5. Node with IRI `https://greggkellogg.net/`

In RDF/XML, the sequence of 5 nodes and predicate arcs on the left-hand side of Figure 2 corresponds to the usage of five XML elements of two types, for the graph nodes and predicate arcs. These are conventionally called ***node elements*** and ***property elements*** respectively. In the striping shown in Example 1, `rdf:Description` is the node element (used three times for the three nodes) and `ex:editor` and `ex:homePage` are the two property elements.

EXAMPLE 1: Striped RDF/XML (nodes and predicate arcs)

```
<rdf:Description>
<ex:editor>
  <rdf:Description>
    <ex:homePage>
      <rdf:Description>
      </rdf:Description>
    </ex:homePage>
  </rdf:Description>
</ex:editor>
</rdf:Description>
```

The Figure 2 graph consists of some nodes that are IRIs (and others that are not, called ***blank node elements***) and this can be added to the RDF/XML using the `rdf:about` attribute on node elements to give the result in Example 2:

EXAMPLE 2: Node Elements with IRIs added

```
<rdf:Description rdf:about="https://www.w3.org/TR/rdf12-xml/">
  <ex:editor>
    <rdf:Description>
      <ex:homePage>
        <rdf:Description rdf:about="https://greggkellogg.net/">
        </rdf:Description>
      </ex:homePage>
    </rdf:Description>
  </ex:editor>
</rdf:Description>
```

Adding the other two paths through the Figure 1 graph to the RDF/XML in Example 2 gives the result in Example 3 (this example fails to show that the blank node is shared between the two paths, see 2.11 Identifying Blank Nodes: `rdf:nodeID`):

EXAMPLE 3: Complete description of all graph paths

```
<rdf:Description rdf:about="https://www.w3.org/TR/rdf12-xml/">
  <ex:editor>
    <rdf:Description>
      <ex:homePage>
        <rdf:Description rdf:about="https://greggkellogg.net/">
        </rdf:Description>
      </ex:homePage>
    </rdf:Description>
  </ex:editor>
</rdf:Description>

<rdf:Description rdf:about="https://www.w3.org/TR/rdf12-xml/">
  <ex:editor>
    <rdf:Description>
      <ex:fullName>Gregg Kellogg</ex:fullName>
    </rdf:Description>
  </ex:editor>
</rdf:Description>

<rdf:Description rdf:about="https://www.w3.org/TR/rdf12-xml/">
  <dc:title>RDF 1.2 XML Syntax</dc:title>
</rdf:Description>
```

§ 2.3 Multiple Property Elements

There are several abbreviations that can be used to make common uses easier to write down. In particular, it is common that a subject node in the RDF graph has multiple outgoing predicate arcs. RDF/XML provides an abbreviation for the corresponding syntax when a node element about a resource has multiple property elements. This can be abbreviated by using multiple child property elements inside the node element describing the subject node.

Taking Example 3, there are two node elements that can take multiple property elements. The subject node with IRI `https://www.w3.org/TR/rdf12-xml/` has property elements

ex:editor and dc:title and the node element for the blank node can take ex:homePage and ex:fullName. This abbreviation gives the result shown in Example 4 (this example does show that there is a single blank node):

EXAMPLE 4: Using multiple property elements on a node element

```
<rdf:Description rdf:about="https://www.w3.org/TR/rdf12-xml/">
  <ex:editor>
    <rdf:Description>
      <ex:homePage>
        <rdf:Description rdf:about="https://greggkellogg.net/">
        </rdf:Description>
      </ex:homePage>
      <ex:fullName>Gregg Kellogg</ex:fullName>
    </rdf:Description>
  </ex:editor>
  <dc:title>RDF 1.2 XML Syntax</dc:title>
</rdf:Description>
```

§ 2.4 Empty Property Elements

When a predicate arc in an RDF graph points to an object node which has no further predicate arcs, which appears in RDF/XML as an empty node element <rdf:Description rdf:about="..."> </rdf:Description> (or <rdf:Description rdf:about="..." />) this form can be shortened. This is done by using the IRI of the object node as the value of an XML attribute rdf:resource on the containing property element and making the property element empty.

In this example, the property element ex:homePage contains an empty node element with the IRI https://greggkellogg.net/. This can be replaced with the empty property element form giving the result shown in Example 5:

EXAMPLE 5: Empty property elements

```
<rdf:Description rdf:about="https://www.w3.org/TR/rdf12-xml/">
  <ex:editor>
    <rdf:Description>
      <ex:homePage rdf:resource="https://greggkellogg.net/"/>
      <ex:fullName>Gregg Kellogg</ex:fullName>
    </rdf:Description>
  </ex:editor>
  <dc:title>RDF 1.2 XML Syntax</dc:title>
</rdf:Description>
```

§ 2.5 Property Attributes

When a property element's content is string literal, it may be possible to use it as an XML attribute on the containing node element. This can be done for multiple properties on the same node element only if the property element name is not repeated (required by XML — attribute names are unique on an XML element) and any in-scope `xml:lang` on the property element's string literal (if any) are the same (see 2.7 Languages: `xml:lang`) This abbreviation is known as a ***Property Attribute*** and can be applied to any node element.

This abbreviation can also be used when the property element is `rdf:type` and it has an `rdf:resource` attribute the value of which is interpreted as a IRI object node.

In Example 5:, there are two property elements with string literal content, the `dc:title` and `ex:fullName` property elements. These can be replaced with property attributes giving the result shown in Example 6:

EXAMPLE 6: Replacing property elements with string literal content into property attributes

```
<rdf:Description rdf:about="https://www.w3.org/TR/rdf12-xml/"
           dc:title="RDF 1.2 XML Syntax">
  <ex:editor>
    <rdf:Description ex:fullName="Gregg Kellogg">
      <ex:homePage rdf:resource="https://greggkellogg.net/"/>
    </rdf:Description>
  </ex:editor>
</rdf:Description>
```

§ 2.6 Completing the Document: Document Element and XML Declaration

To create a complete RDF/XML document, the serialization of the graph into XML is usually contained inside an `rdf:RDF` XML element which becomes the top-level XML document element. Conventionally the `rdf:RDF` element is also used to declare the XML namespaces that are used, although that is not required. When there is only one top-level node element inside `rdf:RDF`, the `rdf:RDF` can be omitted although any XML namespaces *MUST* still be declared.

The XML specification also permits an XML declaration at the top of the document with the XML version and possibly the XML content encoding. This is optional but recommended.

Completing the RDF/XML could be done for any of the correct complete graph examples from Example 4 onwards but taking the smallest Example 6 and adding the final components, gives a complete RDF/XML representation of the original Figure 1 graph in Example 7:

EXAMPLE 7: Complete RDF/XML description of Figure 1

Description of Figure 1 graph (example07.rdf, output example07.nt).

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
            xmlns:dc="http://purl.org/dc/elements/1.1/"
            xmlns:ex="http://example.org/stuff/1.0/">

  <rdf:Description rdf:about="https://www.w3.org/TR/rdf12-xml/"
             dc:title="RDF 1.2 XML Syntax">
    <ex:editor>
      <rdf:Description ex:fullName="Gregg Kellogg">
        <ex:homePage rdf:resource="https://greggkellogg.net/" />
      </rdf:Description>
    </ex:editor>
  </rdf:Description>

</rdf:RDF>
```

It is possible to omit `rdf:RDF` in Example 7 above since there is only one `rdf:Description` inside `rdf:RDF` but this is not shown here.

§ 2.7 Languages: `xml:lang`

RDF/XML permits the use of the `xml:lang` attribute as defined by 2.12 Language Identification of XML 1.1 [*XML11*] to allow the identification of content language. The `xml:lang` attribute can be used on any node element or property element to indicate that the included content is in the given language. Typed literals which includes XML literals are not affected by this attribute. The most specific in-scope language present (if any) is applied to property element string literal content or property attribute values. The `xml:lang=""` form indicates the absence of a language identifier.

Some examples of marking content languages for RDF properties are shown in Example 8:

EXAMPLE 8: Complete example of xml:lang

Complete example of `xml:lang` (example08.rdf, output example08.nt)

```
<?xml version="1.0" encoding="utf-8"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:dc="http://purl.org/dc/elements/1.1/">

  <rdf:Description rdf:about="https://www.w3.org/TR/rdf12-xml/">
    <dc:title>RDF 1.2 XML Syntax</dc:title>
    <dc:title xml:lang="en">RDF 1.2 XML Syntax</dc:title>
    <dc:title xml:lang="en-US">RDF 1.2 XML Syntax</dc:title>
  </rdf:Description>

  <rdf:Description rdf:about="http://example.org/buecher/baum" xml:
    <dc:title>Der Baum</dc:title>
    <dc:description>Das Buch ist außergewöhnlich</dc:description>
    <dc:title xml:lang="en">The Tree</dc:title>
  </rdf:Description>

</rdf:RDF>
```

§ 2.8 Base Direction: `its:dir`

RDF 1.2 introduces the concept of a base direction which can be used to specify the initial text direction of language-tagged strings to create a directional language-tagged strings. This is specified by adding the `its:dir` attribute on any node element or property element, where `its` is the typical prefix used for the ITS namespace `http://www.w3.org/2005/11/its`. The supported values for `its:dir` are `"ltr"`, and `"rtl"` as required for directional language-tagged strings.

Some examples of marking base direction for RDF properties are shown in Example 9:

EXAMPLE 9: Complete example of xml:lang with its:dir

Complete example of `its:dir` (example09.rdf, output example09.nt)

```
<?xml version="1.0" encoding="utf-8"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
         xmlns:show="http://example.org/vocab/show/"
         xmlns:its="http://www.w3.org/2005/11/its"
         rdf:version="1.2"
         its:version="2.0">
  <rdf:Description rdf:about="http://example.org/vocab/show/218">
    <rdfs:label>That Seventies Show</rdfs:label>
    <show:localName xml:lang="en">That Seventies Show</show:localNa
    <show:unrelatedName xml:lang="he" its:dir="rtl">HTML שפת סימון
    <show:localName xml:lang="en-US" its:dir="ltr">That Seventies S
  </rdf:Description>
</rdf:RDF>
```

NOTE

The `its:dir` attribute is specifically tied to version 2.0 of [*ITS*], as indicated by the `its:version="2.0"` on the top-level XML document element.

§ 2.9 XML Literals: `rdf:parseType="Literal"`

RDF allows XML literals [RDF12-CONCEPTS] to be given as the object node of a predicate. These are written in RDF/XML as content of a property element (not a property attribute) and indicated using the `rdf:parseType="Literal"` attribute on the containing property element.

An example of writing an XML literal is given in Example 10 where there is a single RDF triple with the subject node IRI `http://example.org/item01`, the predicate IRI

`http://example.org/stuff/1.0/prop` (from `ex:prop`) and the object node with XML literal content beginning `a:Box`.

EXAMPLE 10: Complete example of rdf:parseType=Literal

Complete example of `rdf:parseType="Literal"` (example10.rdf, output example10.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:ex="http://example.org/stuff/1.0/">

  <rdf:Description rdf:about="http://example.org/item01">
    <ex:prop rdf:parseType="Literal" xmlns:a="http://example.org/a#
      <a:Box required="true">
        <a:widget size="10" />
        <a:grommit id="23" />
      </a:Box>
    </ex:prop>
  </rdf:Description>

</rdf:RDF>
```

§ 2.10 Typed Literals: `rdf:datatype`

RDF allows typed literals to be given as the object node of a predicate. Typed literals consist of a literal string and a datatype IRI. These are written in RDF/XML using the same syntax for literal string nodes in the property element form (not property attribute) but with an additional `rdf:datatype="`*datatypeURI*`"` attribute on the property element. Any IRI can be used in the attribute.

An example of an RDF typed literal is given in Example 11 where there is a single RDF triple with the subject node IRI `http://example.org/item01`, the predicate IRI `http://example.org/stuff/1.0/size` (from `ex:size`) and the object node with the

typed literal ("123", `http://www.w3.org/2001/XMLSchema#int`) to be interpreted as an XML Schema [*XMLSCHEMA11-2*] datatype `int`.

EXAMPLE 11: Complete example of rdf:datatype

Complete example of `rdf:datatype` (example11.rdf, output example11.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
            xmlns:ex="http://example.org/stuff/1.0/">

  <rdf:Description rdf:about="http://example.org/item01">
    <ex:size rdf:datatype="http://www.w3.org/2001/XMLSchema#int">12
  </rdf:Description>

</rdf:RDF>
```

§ 2.11 Identifying Blank Nodes: `rdf:nodeID`

Blank nodes in the RDF are distinct but have no IRI. It is sometimes required that the same blank node is referred to in the RDF/XML in multiple places, such as at the subject and object of several RDF triples. In this case, a blank node identifier can be given to the blank node for identifying it in the document. Blank node identifiers in RDF/XML are scoped to the containing XML Information Set [*XML-INFOSET*] document information item. A blank node identifier is used on a node element to replace `rdf:about="`*IRI*`"` or on a property element to replace `rdf:resource="`*IRI*`"` with `rdf:nodeID="`*blank node identifier*`"` in both cases.

Taking Example 7 and explicitly giving a blank node identifier of `abc` to the blank node in it gives the result shown in Example 12. The second `rdf:Description` property element is about the blank node.

EXAMPLE 12: Complete RDF/XML description of graph using rdf:nodeID

Complete RDF/XML description of graph using `rdf:nodeID` identifying the blank node (example12.rdf, output example12.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
            xmlns:dc="http://purl.org/dc/elements/1.1/"
            xmlns:ex="http://example.org/stuff/1.0/">

  <rdf:Description rdf:about="https://www.w3.org/TR/rdf12-xml/"
             dc:title="RDF 1.2 XML Syntax">
    <ex:editor rdf:nodeID="abc"/>
  </rdf:Description>

  <rdf:Description rdf:nodeID="abc" ex:fullName="Gregg Kellogg">
    <ex:homePage rdf:resource="https://greggkellogg.net/"/>
  </rdf:Description>

</rdf:RDF>
```

§ 2.12 Omitting Blank Nodes: `rdf:parseType="Resource"`

Blank nodes (not IRI nodes) in RDF graphs can be written in a form that allows the `<rdf:Description>` `</rdf:Description>` pair to be omitted. The omission is done by putting an `rdf:parseType="Resource"` attribute on the containing property element that turns the property element into a property-and-node element, which can itself have both property elements and property attributes. Property attributes and the `rdf:nodeID` attribute are not permitted on property-and-node elements.

Taking the earlier Example 7, the contents of the `ex:editor` property element could be alternatively done in this fashion to give the form shown in Example 13:

EXAMPLE 13: Complete example using rdf:parseType=Resource

Complete example using `rdf:parseType="Resource"` (example13.rdf, output: example13.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
            xmlns:dc="http://purl.org/dc/elements/1.1/"
            xmlns:ex="http://example.org/stuff/1.0/">
  <rdf:Description rdf:about="https://www.w3.org/TR/rdf12-xml/"
                   dc:title="RDF 1.2 XML Syntax">
    <ex:editor rdf:parseType="Resource">
      <ex:fullName>Gregg Kellogg</ex:fullName>
      <ex:homePage rdf:resource="https://greggkellogg.net/"/>
    </ex:editor>
  </rdf:Description>
</rdf:RDF>
```

§ 2.13 Omitting Nodes: Property Attributes on an empty Property Element

If all of the property elements on a blank node element have string literal values with the same in-scope `xml:lang` value (if present) and each of these property elements appears at most once and there is at most one `rdf:type` property element with a IRI object node, these can be abbreviated by moving them to be property attributes on the containing property element which is made an empty element.

Taking the earlier Example 5, the `ex:editor` property element contains a blank node element with two property elements `ex:fullname` and `ex:homePage`. `ex:homePage` is not suitable here since it does not have a string literal value, so it is being *ignored* for the purposes of this example. The abbreviated form removes the `ex:fullName` property element and adds a new property attribute `ex:fullName` with the string literal value of the deleted property element to the `ex:editor` property element. The blank node element becomes implicit in the now empty `ex:editor` property element. The result is shown in Example 14.

EXAMPLE 14: Complete example of property attributes on an empty property element

Complete example of property attributes on an empty property element (example14.rdf, output example14.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
            xmlns:dc="http://purl.org/dc/elements/1.1/"
            xmlns:ex="http://example.org/stuff/1.0/">

  <rdf:Description rdf:about="https://www.w3.org/TR/rdf12-xml/"
            dc:title="RDF 1.2 XML Syntax">
    <ex:editor ex:fullName="Gregg Kellogg" />
            <!-- Note the ex:homePage property has been ignored for
  </rdf:Description>

</rdf:RDF>
```

§ 2.14 Typed Node Elements

It is common for RDF graphs to have `rdf:type` predicates from subject nodes. These are conventionally called *typed nodes* in the graph, or *typed node elements* in the RDF/XML. RDF/XML allows such triples to be expressed more concisely by replacing the `rdf:Description` node element name with the namespaced-element corresponding to the IRI of the value of the type relationship. There may, of course, be multiple `rdf:type` predicates but only one can be used in this way, the others *MUST* remain as property elements or property attributes.

The typed node elements are commonly used in RDF/XML with the built-in classes in the RDF Vocabulary: `rdf:Seq`, `rdf:Bag`, `rdf:Alt`, `rdf:Statement`, `rdf:Property` and `rdf:List`.

For example, the RDF/XML in 15 could be written as shown in Example 16.

EXAMPLE 15

Complete example with `rdf:type` (example15.rdf, output example15.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
            xmlns:dc="http://purl.org/dc/elements/1.1/"
            xmlns:ex="http://example.org/stuff/1.0/">

  <rdf:Description rdf:about="http://example.org/thing">
    <rdf:type rdf:resource="http://example.org/stuff/1.0/Document"/
    <dc:title>A marvelous thing</dc:title>
  </rdf:Description>
</rdf:RDF>
```

EXAMPLE 16

Complete example using a typed node element to replace an `rdf:type` (example16.rdf, output example16.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
            xmlns:dc="http://purl.org/dc/elements/1.1/"
            xmlns:ex="http://example.org/stuff/1.0/">

  <ex:Document rdf:about="http://example.org/thing">
    <dc:title>A marvelous thing</dc:title>
  </ex:Document>

</rdf:RDF>
```

§ 2.15 Abbreviating IRIs: `rdf:ID` and `xml:base`

RDF/XML allows further abbreviating IRIs in XML attributes in two ways. The XML Infoset provides a base URI attribute `xml:base` that sets the base IRI for resolving relative IRI references, otherwise the base IRI is that of the document. The base IRI applies to all RDF/XML attributes that deal with IRIs which are `rdf:about`, `rdf:resource`, `rdf:ID` and `rdf:datatype`.

The `rdf:ID` attribute on a node element (not on a property element; there it has another meaning) can be used instead of `rdf:about` and gives a relative IRI reference equivalent to `#` concatenated with the `rdf:ID` attribute value. So for example if `rdf:ID="name"`, that would be equivalent to `rdf:about="#name"`. `rdf:ID` provides an additional check since the same *name* can only appear once in the scope of an `xml:base` value (or document, if none is given), so is useful for defining a set of distinct, related terms relative to the same IRI.

Both forms require a base IRI to be determined, either from an in-scope `xml:base` or from the URL of the RDF/XML document.

Example 17 shows abbreviating the node IRI of `http://example.org/here/#snack` using an `xml:base` of `http://example.org/here/` and an `rdf:ID` on the `rdf:Description` node element. The object node of the `ex:prop` predicate is an IRI resolved from the `rdf:resource` XML attribute value using the in-scope base IRI to give the IRI `http://example.org/here/fruit/apple`.

EXAMPLE 17: Complete example using rdf:ID> and xml:base

Complete example using `rdf:ID` and `xml:base` for shortening IRIs (example17.rdf, output example17.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
            xmlns:ex="http://example.org/stuff/1.0/"
            xml:base="http://example.org/here/">

  <rdf:Description rdf:ID="snack">
    <ex:prop rdf:resource="fruit/apple"/>
  </rdf:Description>

</rdf:RDF>
```

§ 2.16 Container Membership Property Elements: `rdf:li` and `rdf:_n`

RDF has a set of container membership properties and corresponding property elements that are mostly used with instances of the `rdf:Seq`, `rdf:Bag`, and `rdf:Alt` classes which may be written as typed node elements. The list properties are `rdf:_1`, `rdf:_2` etc. and can be written as property elements or property attributes as shown in Example 18. There is an `rdf:li` special property element that is equivalent to `rdf:_1`, `rdf:_2` in order, explained in detail in 6.4 List Expansion Rules. The mapping to the container membership properties is always done in the order that the `rdf:li` special property elements appear in XML — the document order is significant. The equivalent RDF/XML to Example 18 written in this form is shown in Example 19.

EXAMPLE 18: Complex example using RDF list properties

Complex example using RDF list properties (example18.rdf, output example18.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">

  <rdf:Seq rdf:about="http://example.org/favourite-fruit">
    <rdf:_1 rdf:resource="http://example.org/banana"/>
    <rdf:_2 rdf:resource="http://example.org/apple"/>
    <rdf:_3 rdf:resource="http://example.org/pear"/>
  </rdf:Seq>

</rdf:RDF>
```

EXAMPLE 19: Complete example using rdf:li

Complete example using `rdf:li` property element for list properties (example19.rdf, output example19.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">

  <rdf:Seq rdf:about="http://example.org/favourite-fruit">
    <rdf:li rdf:resource="http://example.org/banana"/>
    <rdf:li rdf:resource="http://example.org/apple"/>
    <rdf:li rdf:resource="http://example.org/pear"/>
  </rdf:Seq>

</rdf:RDF>
```

§ 2.17 Collections: `rdf:parseType="Collection"`

RDF/XML allows an `rdf:parseType="Collection"` attribute on a property element to let it contain multiple node elements. These contained node elements give the set of subject nodes of the collection. This syntax form corresponds to a set of triples connecting the collection of subject nodes, the exact triples generated are described in detail in 6.2.20 Production parseTypeCollectionPropertyElt. The collection construction is always done in the order that the node elements appear in the XML document. Whether the order of the collection of nodes is significant is an application issue and not defined here.

Example 20 shows a collection of three nodes elements at the end of the `ex:hasFruit` property element using this form.

EXAMPLE 20: Complete example of a RDF collection

Complete example of a RDF collection of nodes using `rdf:parseType="Collection"` (example20.rdf, output example20.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
            xmlns:ex="http://example.org/stuff/1.0/">

  <rdf:Description rdf:about="http://example.org/basket">
    <ex:hasFruit rdf:parseType="Collection">
      <rdf:Description rdf:about="http://example.org/banana"/>
      <rdf:Description rdf:about="http://example.org/apple"/>
      <rdf:Description rdf:about="http://example.org/pear"/>
    </ex:hasFruit>
  </rdf:Description>

</rdf:RDF>
```

§ 2.18 Reifying Statements: `rdf:ID`

The `rdf:ID` attribute can be used on a property element to reify the triple that it generates (See 6.3 Reification Rules for the full details). The identifier for the triple should be constructed as an IRI made from the relative IRI reference `#` concatenated with the `rdf:ID` attribute value, resolved against the in-scope base IRI. So for example if `rdf:ID="triple"`, that would be equivalent to the IRI formed from relative IRI reference `#triple` against the base IRI. Each (`rdf:ID` attribute value, base IRI) pair has to be unique in an RDF/XML document, see constraint-id.

Example 21 shows a `rdf:ID` being used to reify a triple made from the `ex:prop` property element giving the reified triple the IRI `http://example.org/triples/#triple1`.

EXAMPLE 21

Complete example of `rdf:ID` reifying a property element (example21.rdf, output example21.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
            xmlns:ex="http://example.org/stuff/1.0/"
            xml:base="http://example.org/triples/">
  <rdf:Description rdf:about="http://example.org/">
    <ex:prop rdf:ID="triple1">blah</ex:prop>
  </rdf:Description>

</rdf:RDF>
```

§ 2.19 Triple Terms: `rdf:parseType="Triple"`

A triple term may be the object of an RDF triple. triple terms in RDF graphs can be written in a form that allows a triple term to be written as a resource encoding a single RDF triple. This is done by adding the `rdf:parseType="Triple"` attribute on a property element to describe a single triple term, which may not appear as a triple within the graph, itself; see 6.2.19 Production parseTypeTriplePropertyElt.

EXAMPLE 22: Complete example using rdf:parseType=Triple

Complete example using `rdf:parseType="Triple"` (example22.rdf, output: example22.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:ex="http://example.org/stuff/1.0/"
         xml:base="http://example.org/triples/"
         rdf:version="1.2">
  <rdf:Description rdf:about="http://example.org/">
    <ex:prop rdf:parseType="Triple">
      <rdf:Description rdf:about="http://example.org/stuff/1.0/s">
        <ex:p rdf:resource="http://example.org/stuff/1.0/o" />
      </rdf:Description>
    </ex:prop>
  </rdf:Description>
</rdf:RDF>
```

NOTE

Triple terms play a similar role to Reifying Statements, the difference being that triple terms are are a core part of the RDF 1.2 Abstract Syntax [*RDF12-CONCEPTS*], while Reifying Statements emit triples to describe another triple in the asserted graph. Triple terms describe triples that are not necessarily asserted. See 2.20 Annotating Triples: `rdf:annotation` or `rdf:annotationNodeID` for a way to combine asserting and describing a triple at the same time.

§ 2.20 Annotating Triples: `rdf:annotation` or `rdf:annotationNodeID`

RDF 1.2 XML Syntax also includes syntax for ***annotating*** triples by both asserting the triple, via an explicit identifier, and having the reifier associated with that triple terms be the subject or object of further triples. If explicitly identified, the same reifier can then be used as the subject or object of additional triples and/or triple terms.

A reifier can be added as a property attribute using either the `rdf:annotation` (for IRI reifier) or `rdf:annotationNodeID` (for blank node reifier). This reifier can also be used as the subject of other triples as a way to associate those triples with the triple term composed of the subject, predicate, and object associated with a property element.

EXAMPLE 23: Complete example using rdf:annotation

Complete example using `rdf:annotation` (example23.rdf, output: example23.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:ex="http://example.org/stuff/1.0/"
         xml:base="http://example.org/triples/">
  <rdf:Description rdf:about="http://example.org/">
    <ex:prop rdf:annotation="http://example.org/triple1">blah</ex:p
  </rdf:Description>
  <rdf:Description rdf:about="http://example.org/triple1">
    <ex:prop>foo</ex:prop>
  </rdf:Description>
</rdf:RDF>
```

EXAMPLE 24: Complete example using rdf:annotationNodeID

Complete example using `rdf:annotationNodeID` (example24.rdf, output: example24.nt)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:ex="http://example.org/stuff/1.0/"
         xml:base="http://example.org/triples/">
  <rdf:Description rdf:about="http://example.org/">
    <ex:prop rdf:annotationNodeID="triple1">blah</ex:prop>
  </rdf:Description>
  <rdf:Description rdf:nodeID="triple1">
    <ex:prop>foo</ex:prop>
  </rdf:Description>
</rdf:RDF>
```

§ 3. Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key words *MAY*, *MUST*, *REQUIRED*, *SHOULD*, and *SHOULD NOT* in this document are to be interpreted as described in BCP 14 [*RFC2119*] [*RFC8174*] when, and only when, they appear in all capitals, as shown here.

All use of string without further qualification refers to a Unicode [*UNICODE*] character string; a sequence of characters represented by a code point in Unicode.

Definition:

An ***RDF/XML Document*** is an RDF Document written in the XML syntax for RDF as defined in this document.

Conformance:

An RDF/XML Document is a ***conforming RDF/XML document*** if it adheres to the specification defined in this document.

§ 3.1 RDF/XML Internet Media Type, File Extension, and Macintosh File Type

Contact:
Dan Brickley

See also:
How to Register a Media Type for a W3C Specification

Internet Media Type registration, consistency of use
TAG Finding 3 June 2002 (Revised 4 September 2002)

The Internet Media Type (formerly known as MIME Type) for RDF/XML is `application/rdf+xml` — [*RFC3870*].

It is recommended that RDF/XML files have the extension ".rdf" (all lowercase) on all platforms.

It is recommended that RDF/XML files stored on Macintosh HFS file systems be given a file type of `"rdf "` (all lowercase, with a space character as the fourth letter).

Features introduced in RDF 1.2 require a version announcement on an in-scope node element with an `rdf:version` attribute with a defined value other than `"1.1"`. Acceptable values for `rdf:version` are defined in 2.1 Version Labels in [*RDF12-CONCEPTS*]. Parser behavior for version values is otherwise undefined and *MAY* be used to signal errors or warnings.

The `application/rdf+xml` media type has been registered at IANA as [*RFC3870*].

§ 3.2 Privacy Considerations

This section is non-normative.

The RDF/XML format is used to express arbitrary application data, which may include the expression of personally identifiable information (PII) or other information which could be

considered sensitive. Authors publishing such information are advised to carefully consider the needs and use of publishing such information, as well as the applicable regulations for the regions where the data is expected to be consumed and potentially revealed (e.g., GDPR, CCPA, others), particularly whether authorization measures are needed for access to the data.

§ 3.3 Security Considerations

This section is non-normative.

See the Security Considerations Section of [*RFC3870*].

§ 4. Global Issues

§ 4.1 The RDF Namespace and Vocabulary

The ***RDF namespace IRI*** (or namespace name) is `http://www.w3.org/1999/02/22-rdf-syntax-ns#` and is typically used in XML with the prefix `rdf` although other prefix strings may be used. The RDF Vocabulary is identified by this namespace name and consists of the following names only:

Syntax names — not concepts

`RDF Description ID about annotation annotationNodeID datatype li nodeID parseType resource version`

Class names

`Seq Bag Alt Statement Property XMLLiteral List`

Property names

`subject predicate object type value first rest _`*n*

where *n* is a decimal integer greater than zero with no leading zeros.

Resource names

`nil`

Any other names are not defined and *SHOULD* generate a warning when encountered, but should otherwise behave normally.

Within RDF/XML documents it is not permitted to use XML namespaces whose namespace name is the RDF namespace IRI concatenated with additional characters.

Throughout this document the terminology `rdf:`*name* will be used to indicate *name* is from the RDF Vocabulary and it has a IRI of the concatenation of the RDF namespace IRI and *name*. For example, `rdf:type` has the IRI `http://www.w3.org/1999/02/22-rdf-syntax-ns#type`

§ 4.2 Identifiers

The RDF Concepts document [*RDF12-CONCEPTS*] defines the four types of RDF data that can act as node and/or predicate:

IRI

IRIs can act as node (both subject and object) and as predicate.

IRIs can be either:

- given as XML attribute values interpreted as relative IRI references that are resolved against the in-scope base IRI as described in 4.3 Resolving IRIs to give resolved IRIs
- transformed from XML namespace-qualified element and attribute names (QNames)
- transformed from `rdf:ID` attribute values.

Within RDF/XML, XML QNames are transformed into IRIs by appending the XML local name to the namespace name (IRI). For example, if the XML namespace prefix `foo` has namespace name (IRI) `http://example.org/somewhere/` then the QName `foo:bar` would correspond to the IRI `http://example.org/somewhere/bar`. Note that this restricts which IRIs can be made and the same IRI can be given in multiple ways.

The `rdf:ID` values are transformed into IRIs by appending the attribute value to the result of appending "#" to the in-scope base IRI which is defined in 4.3 Resolving IRIs

Literal

Literals can only act as object nodes.

Literals always have a datatype. Language-tagged strings get the datatype `rdf:langString`. When there is no language tag or datatype specified, the literal is treated as if the datatype `xsd:string` was specified.

Blank Node

Blank nodes can act as subject node and as object node.

Blank nodes have distinct identity in the RDF graph. When the graph is written in a syntax such as RDF/XML, these blank nodes may need document-local identifiers and a syntax in order to preserve this distinction. These local identifiers are called blank node identifiers and are used in RDF/XML as values of the `rdf:nodeID` attribute with the syntax given in Production nodeIdAttr. Blank node identifiers in RDF/XML are scoped to the XML Information Set [*XML-INFOSET*] document information item.

If no blank node identifier is given explicitly as an `rdf:nodeID` attribute value then one will need to be generated (using generated-blank-node-id, see 5.3.3 Grammar Action Notation). Such generated blank node identifiers *MUST* not clash with any blank node identifiers derived from `rdf:nodeID` attribute values. This can be implemented by any method that preserves the distinct identity of all the blank nodes in the graph, that is, the same blank node identifier is not given for different blank nodes. One possible method would be to add a constant prefix to all the `rdf:nodeID` attribute values and ensure no generated blank node identifiers ever used that prefix. Another would be to map all `rdf:nodeID` attribute values to new generated blank node identifiers and perform that mapping on all such values in the RDF/XML document.

Triple Term

Triple terms can only act as object nodes.

A triple term describes a triple, which may, or may not, be asserted in the graph.

§ 4.3 Resolving IRIs

RDF/XML supports XML Base [*XMLBASE*] which defines a ·base-iri· accessor for each ·root event· and ·element event·. Relative IRI references are resolved into IRIs according to the algorithm specified in [*XMLBASE*] and as per *Uniform Resource Identifier (URI): Generic Syntax* [*RFC3986*] using only the basic algorithm in section 5.2. These specifications do not

specify an algorithm for resolving a fragment identifier alone, such as `#foo`, or the empty string `""` into an IRI. In RDF/XML, a fragment identifier is transformed into an IRI by appending the fragment identifier to the in-scope base IRI. The empty string is transformed into an IRI by substituting the in-scope base IRI.

An empty same document reference "" resolves against the URI part of the base IRI; any fragment part is ignored. See Uniform Resource Identifiers (URI) [*RFC3986*].

> NOTE
>
> **Implementation Note (Informative):** When using a hierarchical base URI that has no path component (/), it must be added before using as a base IRI for resolving.

§ 4.4 Constraints

constraint-id

Each application of production idAttr matches an attribute. The ·rdf-term· accessor of the matched attribute is unique within a single RDF/XML document.

The syntax of the names *MUST* match the rdf-id production.

§ 5. Syntax Data Model

This document specifies the syntax of RDF/XML as a grammar on an alphabet of symbols. The symbols are called *events* in the style of the XPATH Information Set Mapping. A sequence of events is normally derived from an XML document, in which case they are in document order as defined below in 5.2 Information Set Mapping. The sequence these events form are intended to be similar to the sequence of events produced by the [*SAX*] XML API from the same XML document. Sequences of events may be checked against the grammar to determine whether they are or are not syntactically well-formed RDF/XML.

The RDF 1.2 Concepts and Abstract Syntax specification [*RDF12-CONCEPTS*] defines four types of RDF terms: IRIs, literals, blank nodes, and triple terms. Literals are composed of a

lexical form and an optional language tag [*BCP47*] – possibly including a base direction – or datatype IRI. An extra type, `prefix`, is used during parsing to map string identifiers to namespace IRIs.

The grammar productions may include actions which fire when the production is recognized. Taken together these actions define a transformation from any syntactically well-formed RDF/XML sequence of events into a set of triples by mapping matching productions to RDF terms or their components (e.g., language tags, lexical forms of literals). Grammar productions change the parser state and emit triples.

The model given here illustrates one way to create a representation of an RDF Graph from an RDF/XML document. It does not mandate any implementation method — any other method that results in a representation of the same RDF Graph may be used.

In particular:

- This specification permits any representation of an RDF graph; in particular, it does not require the use of N-Triples [*RDF12-N-TRIPLES*].
- This specification does not require the use of [*XPATH*] or [*SAX*]
- This specification places no constraints on the order in which software transforming RDF/XML into a representation of a graph, constructs the representation of the graph.
- Software transforming RDF/XML into a representation of a graph *MAY* eliminate duplicate predicate arcs.

The syntax does not support non-well-formed XML documents, nor documents that otherwise do not have an XML Information Set [*XML-INFOSET*]; for example, that do not conform to Namespaces in XML [*XML-NAMES*].

The Infoset requires support for XML Base [*XMLBASE*]. RDF/XML uses the information item property [base IRI], discussed in 4.3 Resolving IRIs

This specification requires an XML Information Set [*XML-INFOSET*] which supports at least the following information items and properties for RDF/XML:

document information item
: [document element], [children], [base IRI]

element information item
: [local name], [namespace name], [children], [attributes], [parent], [base IRI]

attribute information item

[local name], [namespace name], [normalized value]

character information item

[character code]

There is no mapping of the following items to data model events:

- processing instruction information item
- unexpanded entity reference information item
- comment information item
- document type declaration information item
- unparsed entity information item
- notation information item
- namespace information item

Other information items and properties have no mapping to syntax data model events.

Element information items with reserved XML Names (see Name in *Extensible Markup Language (XML) 1.1 (Second Edition)*) are not mapped to data model element events. These are all those with property [prefix] beginning with `xml` (case independent comparison) and all those with [prefix] property having no value and which have [local name] beginning with `xml` (case independent comparison).

All information items contained inside XML elements matching the parseTypeLiteralPropertyElt production form XML literals and do not follow this mapping. See parseTypeLiteralPropertyElt for further information.

This section is intended to satisfy the requirements for Conformance in the [*XML-INFOSET*] specification. It specifies the information items and properties that are needed to implement this specification.

§ 5.1 Events

There are nine types of event defined in the following subsections. Most events are constructed from an Infoset information item (except for IRI, blank node, literal with no explicit datatype and typed literal). The effect of an event constructor is to create a new event with a unique identity, distinct from all other events. Events have accessor operations on them and most have the *string-value*, *IRI*, *rdf-term*, *literal*, or *bnode* accessors that may be a static value or computed.

§ 5.1.1 Root Event

Constructed from a document information item and takes the following accessors and values.

document-element
: Set to the value of document information item property [document-element].

children
: Set to the value of document information item property [children].

base-iri
: Set to the value of document information item property [base IRI].

language
: Set to the empty string.

direction
: Set to the empty string.

graph
: The graph used for collecting emitted triples, initialized to a new RDF graph.

§ 5.1.2 Element Event

Constructed from an element information item and takes the following accessors and values:

local-name
: Set to the value of element information item property [local name].

namespace-name
: Set to the value of element information item property [namespace name].

children
: Set to the value of element information item property [children].

parent
: Set to the value of element information item property [parent].

base-iri
: Set to the value of element information item property [base IRI].

attributes
: Made from the value of element information item property [attributes] which is a set of attribute information items.

  If this set contains an attribute information item `xml:lang` ([namespace name] property with the value `"http://www.w3.org/XML/1998/namespace"` and [local name] property value "lang") it is removed from the set of attribute information items and the ·language· accessor is set to the [normalized-value] property of the attribute information item.

  If the element has a version accessor with the value `"1.2-basic"`, `"1.2"` or greater, this set contains an attribute information item `its:dir` ([namespace name] property with the value `"http://www.w3.org/2005/11/its"` and [local name] property value "dir") it is removed from the set of attribute information items and the ·direction· accessor is set to the [normalized-value] property of the attribute information item, which *MUST* be one of `"ltr"`, `"rtl"`, or the empty string.

  If the element has a version accessor with the value `"1.2-basic"`, `"1.2"` or greater, this set contains an attribute information item `its:version` ([namespace name] property with the value `"http://www.w3.org/2005/11/its"` and [local name] property value "version") it is removed from the set of attribute information items and the ·its-version· accessor is set to the [normalized-value] property of the attribute information item.

  All remaining reserved XML Names (see Name in *Extensible Markup Language (XML) 1.1 (Second Edition)*) are now removed from the set. These are, all attribute information items in the set with property [prefix] beginning with `xml` (case independent comparison) and all attribute information items with [prefix] property having no value and which have [local name] beginning with `xml` (case independent comparison) are removed. Note that the [base IRI] accessor is computed by XML Base before any `xml:base` attribute information item is deleted.

The remaining set of attribute information items are then used to construct a new set of Attribute Events which is assigned as the value of this accessor.

IRI

Set to an IRI constructed from the concatenation of the value of the namespace-name accessor and the value of the local-name accessor.

li-counter

Set to the integer value 1.

language

Set from the ·attributes· as described above. If no value is given from the attributes, the value is set to the value of the language accessor on the parent event (either a Root Event or an Element Event), which may be the empty string.

direction

Set from the ·attributes· as described above. If no value is given from the attributes, the value is set to the value of the direction accessor on the parent event (either a Root Event or an Element Event), which may be the empty string.

version

Set from the ·attributes· as described above. If no value is given from the attributes, the value is set to the value of the version accessor on the parent event (either a Root Event or an Element Event).

its-version

Set from the ·attributes· as described above. If no value is given from the attributes, the value is set to the value of the version accessor on the parent event (either a Root Event or an Element Event), which *SHOULD* be set to `"2.0"`.

subject

Has no initial value. Takes a value that is an Identifier event, or a triple term. This accessor is used on elements that deal with one node in the RDF graph, this generally being the subject of an RDF triple.

graph

The graph used for collecting emitted triples. The initial value is taken from the parent event (either a Root Event or an Element Event), or a new RDF graph, if not set. May be modified by a `parseType="Triple"` attribute.

§ 5.1.3 End Element Event

Has no accessors. Marks the end of the containing element in the sequence.

§ 5.1.4 Attribute Event

Constructed from an attribute information item and takes the following accessors and values:

local-name
: Set to the value of attribute information item property [local name].

namespace-name
: Set to the value of attribute information item property [namespace name].

string-value
: Set to the value of the attribute information item property [normalized value] as specified by [*XML11*] (if an attribute whose normalized value is a zero-length string, then the string-value is also a zero-length string).

IRI
: If ·namespace-name· is present, set to an IRI constructed from the concatenation of the value of the ·namespace-name· accessor and the value of the ·local-name· accessor. Otherwise if ·local-name· is `ID`, `about`, `parseType`, `resource`, `type`, or `version`, set to an IRI constructed from the value of the concatenation of the RDF namespace IRI and the value of the local-name accessor. Other non-namespaced ·local-name· accessor values are forbidden.

  The support for a limited set of non-namespaced names is *REQUIRED* and intended to allow RDF/XML documents specified in [*RDF-SYNTAX-GRAMMAR-19990222*] to remain valid; new documents *SHOULD NOT* use these unqualified attributes and applications *MAY* choose to warn when the unqualified form is seen in a document.

  The construction of IRIs from XML attributes can generate the same IRIs from different XML attributes. This can cause ambiguity in the grammar when matching attribute events (such as when `rdf:about` and `about` XML attributes are both present). Documents that have this are illegal.

rdf-term
: The value is an IRI constructed from string-value resolved against the ·base-iri· accessor of the containing element.

§ 5.1.5 Text Event

Constructed from a sequence of one or more consecutive character information items. Has the single accessor:

string-value
: Set to the value of the string made from concatenating the [character code] property of each of the character information items.

§ 5.1.6 IRI Event

An event for an IRI which has the following accessors:

identifier
: Takes a string value used as an IRI.

rdf-term
: The value is an IRI constructed from the value of the ·identifier· accessor.

These events are constructed by giving a value for the ·identifier· accessor.

For further information on identifiers in RDF, see 4.2 Identifiers.

§ 5.1.7 Triple Term Event

An event for an triple term which has the following accessors:

rdf-term
: The value is a triple term.

These events are constructed by giving a value to the rdf-term accessor.

§ 5.1.8 Blank Node Identifier Event

An event for a blank node identifier which has the following accessors:

identifier
: Takes a string value.

blank-node
: The value is a blank node with local identifier taken from the ·identifier· accessor. Within an RDF/XML document, all blank nodes identified by the same identifier represent the same blank node.

These events are constructed by giving a value for the ·identifier· accessor.

For further information on identifiers in RDF, see 4.2 Identifiers.

§ 5.1.9 Literal Without Datatype Event

> NOTE
>
> RDF/XML literals without an explicit datatype in RDF 1.2 are treated as syntactic sugar for a literal with datatype `xsd:string` (when no language tag is present), as a literal with datatype `rdf:langString` (when a language tag is present and a direction is not present), or as a literal with datatype `rdf:dirLangString` (when both a language tag and a direction are present). The mapping to N-Triples as defined in this subsection is not affected by this change.

An event for a literal without an explicit datatype which can have the following accessors:

literal-value
: Takes a string value.

literal-language
: Takes a string value used as a language tag in language-tagged string.

literal-direction

Takes a string value used as a direction tag in a directional language-tagged strings.

literal

The value is an RDF literal constructed from the other accessors as follows.

If ·literal-language· is the empty string then the value is an RDF literal with the lexical form taken from the value of the ·literal-value· accessor and datatype IRI `xsd:string`.

Otherwise, if the ·literal-direction· is the empty string, then the value is a language-tagged string with the lexical form taken from the value of the ·literal-value· accessor and the language tag taken from the value of the ·literal-language· accessor.

Otherwise the value is a directional language-tagged string with the lexical form taken from the value of the ·literal-value· accessor, the language tag taken from the value of the ·literal-language· accessor, and the base direction taken from the value of the ·literal-direction· accessor.

These events are constructed by giving values for the ·literal-value·, ·literal-language·, and ·literal-direction· accessors.

> NOTE
>
> **Interoperability Note (Informative):** Literals beginning with a Unicode combining character are allowed however they may cause interoperability problems. See [*CHARMOD*] for further information.

§ 5.1.10 Typed Literal Event

An event for a typed literal which can have the following accessors:

literal-value
Takes a string value.

literal-datatype
Takes a string value used as an IRI.

literal

The value is an RDF literal with the lexical form taken from the value of the ·literal-value· accessor and datatype IRI taken from the value of the ·literal-datatype· accessor.

These events are constructed by giving values for the ·literal-value· and ·literal-datatype· accessors.

NOTE

Interoperability Note (Informative): Literals beginning with a Unicode combining character are allowed however they may cause interoperability problems. See [*CHARMOD*] for further information.

NOTE

Implementation Note (Informative): In XML Schema (part 1) [*XMLSCHEMA11-2*], white space normalization occurs during validation according to the value of the whiteSpace facet. The syntax mapping used in this document occurs after this, so the whiteSpace facet formally has no further effect.

§ 5.2 Information Set Mapping

To transform the Infoset into the sequence of events in *document order*, each information item is transformed as described above to generate a tree of events with accessors and values. Each element event is then replaced as described below to turn the tree of events into a sequence in document order.

1. The original element event
2. The value of the children accessor recursively transformed, a possibly empty ordered list of events.
3. An end element event

§ 5.3 Grammar Notation

The following notation is used to describe matching the sequence of data model events as given in 5. Syntax Data Model and the actions to perform for the matches. The RDF/XML grammar is defined in terms of mapping from these matched data model events to triples, using notation of the form:

> number *event-type event-content*
>
> *action*...
>
> `Triple(s)`

where the *event-content* is an expression matching *event-types* (as defined in 5.1 Events), using notation given in the following sections. The number is used for reference purposes. The grammar *action* may include generating new triples to the graph.

The following sections describe the general notation used and that for event matching and actions.

§ 5.3.1 Grammar General Notation

Notation	Meaning
event.accessor	The value of an event accessor.
`rdf:`*X*	A URI as defined in 4.1 The RDF Namespace and Vocabulary.
"ABC"	A string of characters A, B, C in order.

§ 5.3.2 Grammar Event Matching Notation

Notation	Meaning
A == B	Event accessor A matches expression B.
A != B	A is not equal to B.
A | B | ...	The A, B, ... terms are alternatives.
A - B	The terms in A excluding all the terms in B.
anyURI.	Any URI.

anyString.	Any string.
list(item1, item2, ...); list()	An ordered list of events. An empty list.
set(item1, item2, ...); set()	An unordered set of events. An empty set.
*	Zero or more of preceding term.
?	Zero or one of preceding term.
+	One or more of preceding term.
root(acc1 == value1, acc2 == value2, ...)	Match a Root Event with accessors.
start-element(acc1 == value1, acc2 == value2, ...) *children* end-element()	Match a sequence of Element Event with accessors, a possibly empty list of events as element content and an End Element Event.
attribute(acc1 == value1, acc2 == value2, ...)	Match an Attribute Event with accessors.
text()	Match a Text Event.

§ 5.3.3 Grammar Action Notation

Notation	**Meaning**
A := B	Assigns A the value B.
concat(A, B, ..)	A string created by concatenating the terms in order.
resolve(*e*, *s*)	An IRI created by interpreting *s* as a relative IRI reference to the ·base-iri· accessor of *e* as defined in 4.3 Resolving IRIs.
generated-blank-node-id()	A string value for a new distinct generated blank node identifier as defined in 4.2 Identifiers.

event.accessor := *value*	Sets an event accessor to the given value.
iri(identifier := value)	Create a new IRI Event.
bnodeid(identifier := value)	Create a new Blank Node Identifier Event. See also section 4.2 Identifiers.
literal(literal-value := string, literal-language := language, literal-direction := direction, ...)	Create a new Literal Without Datatype Event.
string(iri)	The string value of an IRI.
typed-literal(literal-value := string, ...)	Create a new Typed Literal Event.
triple(s, p, o)	Create a new Triple Term Event composed of *s*, *p*, and *o*.

§ 6. RDF/XML Grammar

§ 6.1 Grammar summary

6.2.2 Production coreSyntaxTerms	`rdf:RDF` | `rdf:ID` | `rdf:about` | `rdf:annotation` | `rdf:annotationNodeID` | `rdf:parseType` | `rdf:resource` | `rdf:nodeID` | `rdf:datatype` | `rdf:version`
6.2.3 Production syntaxTerms	coreSyntaxTerms | `rdf:Description` | `rdf:li`
6.2.4 Production oldTerms	`rdf:aboutEach` | `rdf:aboutEachPrefix` | `rdf:bagID`
6.2.5 Production nodeElementIRIs	anyURI - (coreSyntaxTerms | `rdf:li` | oldTerms)
6.2.6 Production propertyElementURIs	anyURI - (coreSyntaxTerms | `rdf:Description` | oldTerms)
6.2.7 Production propertyAttributeIRIs	anyURI - (coreSyntaxTerms | `rdf:Description` | `rdf:li` | oldTerms)

6.2.8 Production doc	root(document-element == RDF, children == list(RDF))
6.2.9 Production RDF	start-element(IRI == rdf:RDF, attributes == set()) nodeElementList end-element()
6.2.10 Production nodeElementList	ws* (nodeElement ws*)*
6.2.11 Production nodeElement	start-element(IRI == nodeElementIRIs attributes == set(idAttr | nodeIdAttr | aboutAttr)?, propertyAttr*) propertyEltList end-element()
6.2.12 Production ws	A text event matching white space defined by XML [*XML11*] definition *White Space* Rule [3] S in section Common Syntactic Constructs
6.2.13 Production propertyEltList	ws* (propertyElt ws*) *
6.2.14 Production propertyElt	resourcePropertyElt | literalPropertyElt | parseTypeLiteralPropertyElt | parseTypeResourcePropertyElt | parseTypeTriplePropertyElt | parseTypeCollectionPropertyElt | parseTypeOtherPropertyElt | emptyPropertyElt
6.2.15 Production resourcePropertyElt	start-element(IRI == propertyElementURIs), attributes == set(idAttr?)) ws* nodeElement ws* end-element()
6.2.16 Production literalPropertyElt	start-element(IRI == propertyElementURIs), attributes == set(idAttr?, (annotationAttr | annotationNodeIDAttr)?, datatypeAttr?)) text() end-element()
6.2.17 Production parseTypeLiteralPropertyElt	start-element(IRI == propertyElementURIs), attributes == set(idAttr?, (annotationAttr | annotationNodeIDAttr)?, parseLiteral)) literal end-element()
	start-element(IRI == propertyElementURIs), attributes ==

6.2.18 Production parseTypeResourcePropertyElt	set(idAttr?, (annotationAttr | annotationNodeIDAttr)?, parseResource)) propertyEltList end-element()
6.2.19 Production parseTypeTriplePropertyElt	start-element(IRI == propertyElementURIs), attributes == set(idAttr?, (annotationAttr | annotationNodeIDAttr)?, parseTriple)) propertyEltList end-element()
6.2.20 Production parseTypeCollectionPropertyElt	start-element(IRI == propertyElementURIs), attributes == set(idAttr?, parseCollection)) nodeElementList end-element()
6.2.21 Production parseTypeOtherPropertyElt	start-element(IRI == propertyElementURIs), attributes == set(idAttr?, (annotationAttr | annotationNodeIDAttr)?, parseOther)) propertyEltList end-element()
6.2.22 Production emptyPropertyElt	start-element(IRI == propertyElementURIs), attributes == set(idAttr?, (annotationAttr | annotationNodeIDAttr)?, (resourceAttr | nodeIdAttr | datatypeAttr)?, propertyAttr*)) end-element()
6.2.23 Production idAttr	attribute(IRI == `rdf:ID`, string-value == rdf-id)
6.2.24 Production nodeIdAttr	attribute(IRI == `rdf:nodeID`, string-value == rdf-id)
6.2.25 Production aboutAttr	attribute(IRI == `rdf:about`, string-value == string(IRI))
6.2.28 Production propertyAttr	attribute(IRI == propertyAttributeIRIs, string-value == anyString)
6.2.29 Production resourceAttr	attribute(IRI == `rdf:resource`, string-value == string(IRI))
6.2.30 Production datatypeAttr	attribute(IRI == `rdf:datatype`, string-value == string(IRI))
6.2.31 Production parseLiteral	attribute(IRI == `rdf:parseType`, string-value == "Literal")
6.2.32 Production parseResource	attribute(IRI == `rdf:parseType`, string-value == "Resource")
6.2.33 Production parseTriple	attribute(IRI == `rdf:parseType`, string-value == "Triple")
6.2.34 Production	attribute(IRI == `rdf:parseType`, string-value ==

parseCollection	"Collection")
6.2.35 Production parseOther	attribute(IRI == `rdf:parseType`, string-value == anyString - ("Resource" | "Literal" | "Collection"))
6.2.36 Production IRI	An IRI.
6.2.37 Production literal	Any XML element content that is allowed according to [*XML11*] definition *Content of Elements* Rule [43] content. in section 3.1 Start-Tags, End-Tags, and Empty-Element Tags
6.2.38 Production rdf-id	An attribute ·string-value· matching any legal [*XML-NAMES*] token NCName

§ 6.2 Grammar Productions

§ 6.2.1 Grammar start

If the RDF/XML is a standalone XML document (identified by presentation as an `application/rdf+xml` RDF media type object, or by some other means) then the grammar may start with production doc or production nodeElement.

If the content is known to be RDF/XML by context, such as when RDF/XML is embedded inside other XML content, then the grammar can either start at Element Event RDF (only when an element is legal at that point in the XML) or at production nodeElementList (only when element content is legal, since this is a list of elements). For such embedded RDF/XML, the ·base-iri· value on the outermost element *MUST* be initialized from the containing XML since no Root Event will be available. Note that if such embedding occurs, the grammar may be entered several times but no state is expected to be preserved.

> EDITOR'S NOTE
>
> Add tests for embedded RDF/XML.

The start node is referred to as the ***top-level XML document element***.

§ 6.2.2 Production coreSyntaxTerms

`rdf:RDF` | `rdf:ID` | `rdf:about` | `rdf:parseType` | `rdf:resource` | `rdf:nodeID` | `rdf:datatype` | `rdf:version`

A subset of the syntax terms from the RDF Vocabulary in 4.1 The RDF Namespace and Vocabulary which are used in RDF/XML.

§ 6.2.3 Production syntaxTerms

coreSyntaxTerms | `rdf:Description` | `rdf:li`

All the syntax terms from the RDF Vocabulary in 4.1 The RDF Namespace and Vocabulary which are used in RDF/XML.

§ 6.2.4 Production oldTerms

`rdf:aboutEach` | `rdf:aboutEachPrefix` | `rdf:bagID`

These are the names from the RDF Vocabulary that have been withdrawn from the language. See the resolutions of Issue rdfms-aboutEach-on-object, Issue rdfms-abouteachprefix and Last Call Issue timbl-01 for further information.

§ 6.2.5 Production nodeElementIRIs

anyURI - (coreSyntaxTerms | `rdf:li` | oldTerms)

The IRIs that are allowed on node elements.

§ 6.2.6 Production propertyElementURIs

anyURI - (coreSyntaxTerms | `rdf:Description` | oldTerms)

The URIs that are allowed on property elements.

§ 6.2.7 Production propertyAttributeIRIs

anyURI - (coreSyntaxTerms | `rdf:Description` | `rdf:li` | oldTerms)

The IRIs that are allowed on property attributes.

§ 6.2.8 Production doc

root(document-element == RDF,
  children == list(RDF))

§ 6.2.9 Production RDF

start-element(IRI == `rdf:RDF`,
  attributes == set())
nodeElementList
end-element()

§ 6.2.10 Production nodeElementList

ws* (nodeElement ws*)*

§ 6.2.11 Production nodeElement

start-element(IRI == nodeElementIRIs
  attributes == set(idAttr | nodeIdAttr | aboutAttr)?, propertyAttr*)
propertyEltList
end-element()

For node element *e*, the processing of some of the attributes has to be done before other work such as dealing with children events or other attributes. These can be processed in any order:

- If there is an attribute *a* with *a*.IRI == `rdf:ID`, then *e*.subject := iri(identifier := resolve(*e*, concat("#", *a*.string-value))).
- If there is an attribute *a* with *a*.IRI == `rdf:nodeID`, then *e*.subject := bnodeid(identifier:=*a*.string-value).
- If there is an attribute *a* with *a*.IRI == `rdf:about` then *e*.subject := iri(identifier := *e*.rdf-term).

If *e*.subject is empty, then *e*.subject := bnodeid(identifier := generated-blank-node-id()).

The following can then be performed in any order:

- If *e*.IRI != `rdf:Description` then the following statement is added to *e*.parent.graph:

  ```
  e.subject.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> e.IRI .
  ```

- If there is an attribute *a* in propertyAttr with *a*.IRI == `rdf:type` then *u*:=iri(identifier:=*e*.rdf-term) and the following triple is *e*.parent.graph:

  ```
  e.subject.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> u.rdf-term .
  ```

- For each attribute *a* matching propertyAttr (and not `rdf:type`), the Unicode string *a*.string-value *SHOULD* be in Normal Form C [*NFC*], *o* := literal(literal-value := *a*.string-value, literal-language := *e*.language, literal-direction := *e*.direction) and the following statement is added to *e*.parent.graph:

```
e.subject.rdf-term a.IRI o.literal .
```

- Handle the propertyEltList children events in document order.

§ 6.2.12 Production ws

A text event matching white space defined by [*XML11*] definition *White Space* Rule [3] S in section Common Syntactic Constructs

§ 6.2.13 Production propertyEltList

ws* (propertyElt ws*) *

§ 6.2.14 Production propertyElt

resourcePropertyElt | literalPropertyElt | parseTypeLiteralPropertyElt | parseTypeResourcePropertyElt | parseTypeTriplePropertyElt | parseTypeCollectionPropertyElt | parseTypeOtherPropertyElt | emptyPropertyElt

If element *e* has *e*.IRI = `rdf:li` then apply the list expansion rules on element *e*.parent in 6.4 List Expansion Rules to give a new IRI *u* and *e*.IRI := *u*.

The action of this production *MUST* be done before the actions of any sub-matches (resourcePropertyElt ... emptyPropertyElt). Alternatively the result *MUST* be equivalent to as if it this action was performed first, such as performing as the first action of all of the sub-matches.

§ 6.2.15 Production resourcePropertyElt

start-element(IRI == propertyElementURIs),
attributes == set(idAttr?, (annotationAttr | annotationNodeIDAttr)?)
ws* nodeElement ws*
end-element()

For element *e*, and the single contained nodeElement *n*, first *n MUST* be processed using production nodeElement. Then the following statement is added to *e*.parent.graph:

```
e.parent.subject.rdf-term e.IRI n.subject.rdf-term .
```

If the `rdf:ID` attribute *a* is given, the above statement is reified with *i* := iri(identifier := resolve(*e*, concat("#", *a*.string-value))) using the reification rules in 6.3 Reification Rules and *e*.subject := *i*

If either the `rdf:annotation` or `rdf:annotationNodeID` attributes *a* are given, the statement above is used to create a triple term *t*, and the following statement is added to *e*.parent.graph:

```
a.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#reifies>
t.rdf-term .
```

§ 6.2.16 Production literalPropertyElt

start-element(IRI == propertyElementURIs),
attributes == set(idAttr?, (annotationAttr | annotationNodeIDAttr)?, datatypeAttr?))
text()
end-element()

Note that the empty literal case is defined in production emptyPropertyElt.

For element *e*, and the text event *t*. The Unicode string *t*.string-value *SHOULD* be in Normal Form C [*NFC*]. If the `rdf:datatype` attribute *d* is given then *o* := typed-literal(literal-value := *t*.string-value, literal-datatype := *d*.rdf-term) otherwise *o* := literal(literal-value := *t*.string-value, literal-language := *e*.language, literal-direction := *e*.direction) and the following statement is added to *e*.parent.graph:

```
e.parent.subject.rdf-term e.IRI o.literal .
```

If the `rdf:ID` attribute *a* is given, the above statement is reified with *i* := iri(identifier := resolve(*e*, concat("#", *a*.string-value))) using the reification rules in 6.3 Reification Rules and *e*.subject := *i*.

If either the `rdf:annotation` or `rdf:annotationNodeID` attributes *a* are given, the statement above is used to create a triple term *t*, and the following statement is added to *e*.parent.graph:

```
  a.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#reifies>
t.rdf-term .
```

§ 6.2.17 Production parseTypeLiteralPropertyElt

This section is non-normative.

start-element(IRI == propertyElementURIs),
  attributes == set(idAttr?, (annotationAttr | annotationNodeIDAttr)?, parseLiteral))
literal
end-element()

For element *e* and the literal *l* that is the `rdf:parseType="Literal"` content. *l* is not transformed by the syntax data model mapping into events (as noted in section 5. Syntax Data Model) but remains an XML Infoset of XML Information items.

l is transformed into the lexical form of an XML literal in the RDF graph *x* (a Unicode string) by the following algorithm. This does not mandate any implementation method — any other method that gives the same result may be used.

1. Use *l* to construct an XPath sequence [*XPATH-DATAMODEL-30*].
2. Apply https://www.w3.org/TR/xpath-functions-30/#func-serialize [*XPATH-FUNCTIONS-30*] to this sequence to give an xsd:string *x*.
3. The Unicode string *x* is used as the lexical form of *l*
4. This Unicode string *x* *SHOULD* be in NFC Normal Form C [*NFC*]

Then *o* := typed-literal(literal-value := *x*, literal-datatype := `http://www.w3.org/1999/02/22-rdf-syntax-ns#XMLLiteral`) and the following

statement is added to *e*.parent.graph:

```
e.parent.subject.rdf-term e.IRI o.literal .
```

If the `rdf:ID` attribute *a* is given, the above statement is reified with *i* := iri(identifier := resolve(*e*, concat("#", *a*.string-value))) using the reification rules in 6.3 Reification Rules and *e*.subject := *i*.

If either the `rdf:annotation` or `rdf:annotationNodeID` attributes *a* are given, the statement above is used to create a triple term *t*, and the following statement is added to *e*.parent.graph:

```
  a.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#reifies>
t.rdf-term .
```

§ 6.2.18 Production parseTypeResourcePropertyElt

start-element(IRI == propertyElementURIs),
  attributes == set(idAttr?, (annotationAttr | annotationNodeIDAttr)?, parseResource))
propertyEltList
end-element()

For element *e* with possibly empty element content *c*.

n := bnodeid(identifier := generated-blank-node-id()).

Add the following statement to *e*.parent.graph:

```
e.parent.subject.rdf-term e.IRI n.rdf-term .
```

If the `rdf:ID` attribute *a* is given, the statement above is reified with *i* := iri(identifier := resolve(*e*, concat("#", *a*.string-value))) using the reification rules in 6.3 Reification Rules and *e*.subject := *i*.

If either the `rdf:annotation` or `rdf:annotationNodeID` attributes *a* are given, the statement above is used to create a triple term *t*, and the following statement is added to *e*.parent.graph:

```
  a.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#reifies>
t.rdf-term .
```

If the element content *c* is not empty, then use event *n* to create a new sequence of events as follows:

start-element(IRI := `rdf:Description`,
  subject := *n*,
  attributes := set())
c
end-element()

Then process the resulting sequence using production nodeElement.

§ 6.2.19 Production parseTypeTriplePropertyElt

start-element(IRI == propertyElementURIs),
  attributes == set(parseTriple))
propertyEltList
end-element()

For element *e* with element content *c*.

Create a new sequence of events as follows:

start-element(IRI := nodeElementIRIs,
  attributes := set())
  graph := a new RDF graph)
c
end-element()

Then process the resulting sequence using production nodeElement.

After processing the node sequence, *e*.parent.graph *MUST* be an RDF graph containing a single triple.

e.subject := triple(*s*, *p*, *o*), where *s*, *p*, and *o* are the subject, predicate, and object of the sole triple in *e*.parent.graph.

Add the following statement added to *e*.parent.graph:

```
e.parent.subject.rdf-term e.IRI e.subject.rdf-term .
```

§ 6.2.20 Production parseTypeCollectionPropertyElt

start-element(IRI == propertyElementURIs),
  attributes == set(idAttr?, parseCollection))
nodeElementList
end-element()

For element event *e* with possibly empty nodeElementList *l*. Set *s*:=list().

For each element event *f* in *l*, *n* := bnodeid(identifier := generated-blank-node-id()) and append *n* to *s* to give a sequence of events.

If *s* is not empty, *n* is the first event identifier in *s* and the following statement is added to *e*.parent.graph:

```
e.parent.subject.rdf-term e.IRI n.rdf-term .
```

otherwise the following statement is added to *e*.parent.graph:

```
e.parent.subject.rdf-term e.IRI <http://www.w3.org/1999/02/22-rdf-syntax-ns#nil> .
```

If the `rdf:ID` attribute *a* is given, either of the the above statements is reified with *i* := iri(identifier := resolve(*e*, concat("#", *a*.string-value))) using the reification rules in 6.3 Reification Rules.

If either the `rdf:annotation` or `rdf:annotationNodeID` attributes *a* are given, the statement above is used to create a triple term *t*, and the following statement is added to *e*.parent.graph:

```
  a.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#reifies>
t.rdf-term .
```

If *s* is empty, no further work is performed.

For each event *n* in *s* and the corresponding element event *f* in *l*, the following statement is added to *e*.parent.graph:

```
n.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#first> f.rdf-
term .
```

For each consecutive and overlapping pair of events (*n*, *o*) in *s*, the following statement is added to *e*.parent.graph:

```
n.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#rest> o.rdf-
term .
```

If *s* is not empty, *n* is the last event identifier in *s*, the following statement is added to *e*.parent.graph:

```
n.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#rest>
<http://www.w3.org/1999/02/22-rdf-syntax-ns#nil> .
```

§ 6.2.21 Production parseTypeOtherPropertyElt

start-element(IRI == propertyElementURIs),
  attributes == set(idAttr?, (annotationAttr | annotationNodeIDAttr)?, parseOther))
propertyEltList
end-element()

All `rdf:parseType` attribute values other than the strings "Resource", "Literal", "Collection", or "Triple" are treated as if the value was "Literal". This production matches and acts as if production parseTypeLiteralPropertyElt was matched. No extra triples are generated for other `rdf:parseType` values.

§ 6.2.22 Production emptyPropertyElt

start-element(IRI == propertyElementURIs),
attributes == set(idAttr?, (annotationAttr | annotationNodeIDAttr)?, (resourceAttr | nodeIdAttr | datatypeAttr)?, propertyAttr*))
end-element()

- If there are no attributes **or** only the optional `rdf:ID` attribute *i*, or the optional attributes `rdf:annotation` or `rdf:annotationNodeID` *a*, then *o* := literal(literal-value:="", literal-language := *e*.language, literal-direction := *e*.direction) and the following statement is added to *e*.parent.graph:

  ```
  e.parent.subject.rdf-term e.IRI o.literal .
  ```

  and then if *i* is given, the above statement is reified with iri(identifier := resolve(*e*, concat("#", *i*.string-value))) using the reification rules in 6.3 Reification Rules.

  and then if *a* is given, the statement above is used to create a triple term *t*, and the following statement is added to *e*.parent.graph:

  ```
  a.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#reifies>
  t.rdf-term .
  ```

- Otherwise

  - If `rdf:resource` attribute *i* is present, then *r* := iri(identifier := *i*.rdf-term)
  - If `rdf:nodeID` attribute *i* is present, then *r* := bnodeid(identifier := *i*.string-value)
  - If neither, *r* := bnodeid(identifier := generated-blank-node-id())

  The following are done in any order:

  - For all propertyAttr attributes *a* (in any order)

    - If *a*.IRI == `rdf:type` then *u*:=iri(identifier:=*i*.rdf-term) and the following triple is added to *e*.parent.graph:

```
r.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> u.rdf-term .
```

- Otherwise Unicode string *a*.string-value *SHOULD* be in Normal Form C [*NFC*], *o* := literal(literal-value := *a*.string-value, literal-language := *e*.language, literal-direction := *e*.direction) and the following statement is added to *e*.parent.graph:

```
r.rdf-term a.IRI o.literal .
```

- Add the following statement to the graph:

```
e.parent.subject.rdf-term e.IRI r.rdf-term .
```

and then if `rdf:ID` attribute *i* is given, the above statement is reified with iri(identifier := resolve(*e*, concat("#", *i*.rdf-term))) using the reification rules in 6.3 Reification Rules.

and then if either the `rdf:annotation` or `rdf:annotationNodeID` attributes *a* are given, the statement above is used to create a triple term *t*, and the following statement is added to *e*.parent.graph:

```
a.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#reifies> t.rdf-term .
```

§ 6.2.23 Production idAttr

attribute(IRI == `rdf:ID`,
string-value == rdf-id)

Constraint:: constraint-id applies to the values of `rdf:ID` attributes

§ 6.2.24 Production nodeIdAttr

attribute(IRI == `rdf:nodeID`,
string-value == rdf-id)

§ 6.2.25 Production aboutAttr

attribute(IRI == `rdf:about`,
  string-value == string(IRI))

§ 6.2.26 Production annotationAttr

attribute(IRI == `rdf:annotation`,
  string-value == string(IRI))

This production *MAY* be ignored unless `rdf:version` is `"1.2"` or greater.

§ 6.2.27 Production annotationNodeIDAttr

attribute(IRI == `rdf:annotationNodeID`,
  rdf-term == bnodeid(rdf-id))

This production *MAY* be ignored unless `rdf:version` is `"1.2"` or greater.

§ 6.2.28 Production propertyAttr

attribute(IRI == propertyAttributeIRIs,
  string-value == anyString)

§ 6.2.29 Production resourceAttr

attribute(IRI == `rdf:resource`,
  string-value == string(IRI))

§ 6.2.30 Production datatypeAttr

attribute(IRI == `rdf:datatype`,
  string-value == string(IRI))

§ 6.2.31 Production parseLiteral

attribute(IRI == `rdf:parseType`,
  string-value == "Literal")

§ 6.2.32 Production parseResource

attribute(IRI == `rdf:parseType`,
  string-value == "Resource")

§ 6.2.33 Production parseTriple

attribute(IRI == `rdf:parseType`,
  string-value == "Triple")

§ 6.2.34 Production parseCollection

attribute(IRI == `rdf:parseType`,
  string-value == "Collection")

§ 6.2.35 Production parseOther

attribute(IRI == `rdf:parseType`,
  string-value == anyString - ("Resource" | "Literal" | "Collection"))

§ 6.2.36 Production IRI

An IRI.

§ 6.2.37 Production literal

Any XML element content that is allowed according to XML definition *Content of Elements* Rule [43] content. in section 3.1 Start-Tags, End-Tags, and Empty-Element Tags

The string-value for the resulting event is discussed in 6.2.17 Production parseTypeLiteralPropertyElt.

§ 6.2.38 Production rdf-id

An attribute ·string-value· matching any legal [*XML-NAMES*] token NCName

§ 6.3 Reification Rules

For the given IRI event *r* and the statement with terms *s*, *p* and *o* corresponding to the N-Triples:

```
s p o .
```

add the following statements to the graph:

```
r.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#subject> s .
r.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#predicate> p .
r.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#object> o .
r.rdf-term <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
<http://www.w3.org/1999/02/22-rdf-syntax-ns#Statement> .
```

§ 6.4 List Expansion Rules

For the given element *e*, create a new IRI *u* := concat("http://www.w3.org/1999/02/22-rdf-syntax-ns#_", *e*.li-counter), increment the *e*.li-counter property by 1 and return *u*.

§ 7. Serializing an RDF Graph to RDF/XML

There are some RDF graphs as defined in [*RDF12-CONCEPTS*] that cannot be serialized in RDF/XML. These are the graphs that:

Use property names that cannot be turned into XML namespace-qualified names.
: An XML namespace-qualified name (QName) has restrictions on the legal characters such that not all property URIs can be expressed as these names. It is recommended that implementors of RDF serializers, in order to break a URI into a namespace name and a local name, split it after the last XML non-NCName character, ensuring that the first character of the name is a Letter or '_'. If the URI ends in a non-NCName character then throw a "this graph cannot be serialized in RDF/XML" exception or error.

Use inappropriate reserved names as properties
: For example, a property with the same URI as any of the syntaxTerms production.

NOTE

Implementation Note (Informative): When an RDF graph is serialized to RDF/XML and has an XML Schema Datatype (XSD), it *SHOULD* be written in a form that does not require whitespace processing. XSD support is NOT required by RDF or RDF/XML so this is optional.

§ 8. Using RDF/XML with SVG

This section is non-normative.

There is a standardized approach for associating RDF compatible metadata with SVG — the metadata element which was explicitly designed for this purpose as defined in Section 21 Metadata of the *Scalable Vector Graphics (SVG) 1.0 Specification* [*SVG10*] and *Scalable Vector Graphics (SVG) 1.1 (Second Edition)* [*SVG11*].

This document contains two example graphs in SVG with such embedded RDF/XML inside the metadata element: figure 1 and figure 2.

§ A. Acknowledgments

This section is non-normative.

§ A.1 Acknowledgments the original specification

This section is non-normative.

This document is a product of extended deliberations by the RDF Core working group, whose members have included: Art Barstow (W3C) Dave Beckett (ILRT), Dan Brickley (W3C/ILRT), Dan Connolly (W3C), Jeremy Carroll (Hewlett Packard), Ron Daniel (Interwoven Inc), Bill

dehOra (InterX), Jos De Roo (AGFA), Jan Grant (ILRT), Graham Klyne (Clearswift and Nine by Nine), Frank Manola (MITRE Corporation), Brian McBride (Hewlett Packard), Eric Miller (W3C), Stephen Petschulat (IBM), Patrick Stickler (Nokia), Aaron Swartz (HWG), Mike Dean (BBN Technologies / Verizon), R. V. Guha (Alpiri Inc), Pat Hayes (IHMC), Sergey Melnik (Stanford University), Martyn Horner (Profium Ltd).

This specification also draws upon an earlier RDF Model and Syntax document edited by Ora Lassilla and Ralph Swick, and RDF Schema edited by Dan Brickley and R. V. Guha. RDF and RDF Schema Working group members who contributed to this earlier work are: Nick Arnett (Verity), Tim Berners-Lee (W3C), Tim Bray (Textuality), Dan Brickley (ILRT / University of Bristol), Walter Chang (Adobe), Sailesh Chutani (Oracle), Dan Connolly (W3C), Ron Daniel (DATAFUSION), Charles Frankston (Microsoft), Patrick Gannon (CommerceNet), RV Guha (Epinions, previously of Netscape Communications), Tom Hill (Apple Computer), Arthur van Hoff (Marimba), Renato Iannella (DSTC), Sandeep Jain (Oracle), Kevin Jones, (InterMind), Emiko Kezuka (Digital Vision Laboratories), Joe Lapp (webMethods Inc.), Ora Lassila (Nokia Research Center), Andrew Layman (Microsoft), Ralph LeVan (OCLC), John McCarthy (Lawrence Berkeley National Laboratory), Chris McConnell (Microsoft), Murray Maloney (Grif), Michael Mealling (Network Solutions), Norbert Mikula (DataChannel), Eric Miller (OCLC), Jim Miller (W3C, emeritus), Frank Olken (Lawrence Berkeley National Laboratory), Jean Paoli (Microsoft), Sri Raghavan (Digital/Compaq), Lisa Rein (webMethods Inc.), Paul Resnick (University of Michigan), Bill Roberts (KnowledgeCite), Tsuyoshi Sakata (Digital Vision Laboratories), Bob Schloss (IBM), Leon Shklar (Pencom Web Works), David Singer (IBM), Wei (William) Song (SISU), Neel Sundaresan (IBM), Ralph Swick (W3C), Naohiko Uramoto (IBM), Charles Wicksteed (Reuters Ltd.), Misha Wolf (Reuters Ltd.), Lauren Wood (SoftQuad).

§ A.2 Acknowledgments for RDF 1.1

This section is non-normative.

Gavin Carothers provided the RDF 1.1 update for the Production parseTypeLiteralPropertyElt. Ivan Herman provided valuable comments and reworked Figs 1 and 2.

This specification is a product of extended deliberations by the members of the RDFcore Working Group and the RDF and RDF Schema Working Group.

The following people provided valuable contributions to the document:

- Dan Brickley, W3C/ILRT
- Jeremy Carroll, HP Labs, Bristol
- Graham Klyne, Nine by Nine
- Bijan Parsia, MIND Lab at University of Maryland at College Park

A.3 Acknowledgments for RDF 1.2

This section is non-normative.

In addition to the editors, the following people have contributed to this specification: Andy Seaborne, Denis Ah-Kang, Dominik Tomaszuk, Jerven Bolleman, John Walker, Peter F. Patel-Schneider, Pierre-Antoine Champin, and Ted Thibodeau Jr

Members of the RDF & SPARQL Working Group Group included Vladimir Alexiev, James Anderson, Amin Anjomshoaa, Julián Arenas-Guerrero, Dörthe Arndt, Bilal Ben Mahria, Erich Bremer, Dan Brickley, Kurt Cagle, Sarven Capadisli, Rémi Ceres, Pierre-Antoine Champin, David Chaves-Fraga, Souripriya Das, Daniil Dobriy, Enrico Franconi, Jeffrey Phillips Freeman, Fabien Gandon, Benjamin Goering, Damien Graux, Adrian Gschwend, Olaf Hartig, Timothée Haudebourg, Ian Horrocks, Gregg Kellogg, Mark Kim, Jose Emilio Labra Gayo, Ora Lassila, Richard Lea, Niklas Lindström, Pasquale Lisena, Thomas Lörtsch, Matthew Nguyen, Peter Patel-Schneider, Thomas Pellissier Tanon, Dave Raggett, Jean-Yves ROSSI, Felix Sasaki, Andy Seaborne, Alan Snyder, Stuart Sutton, Ruben Taelman, Ted Thibodeau Jr, Dominik Tomaszuk, Raphaël Troncy, William Van Woensel, Gregory Williams, Jesse Wright, Achille Zappa, and Antoine Zimmermann.

> EDITOR'S NOTE
>
> Recognize members of the Task Force? Not an easy to find list of contributors.

B. Changes between RDF 1.1 and RDF 1.2

This section is non-normative.

- Adds 3.2 Privacy Considerations and 3.3 Security Considerations.
- Updates reference for [*SAX*].
- Removes the statement that the `rdf:HTML` datatype cannot be serialized in RDF/XML.
- Adds 2.8 Base Direction: `its:dir` and `rdf:version`.
- The processing model now emits RDF terms and triples directly, rather than N-Triples [*RDF12-N-TRIPLES*]; this is more consistent with other concrete RDF syntaxes. The names of some event accessors have been updated accordingly.
- Adds 2.19 Triple Terms: `rdf:parseType="Triple"`, 6.2.19 Production parseTypeTriplePropertyElt, and 6.2.33 Production parseTriple to support Triple Terms.
- Adds 2.20 Annotating Triples: `rdf:annotation` or `rdf:annotationNodeID` and rules for creating reifiers for Triple Terms, allowing them to be used as the subject or object of other triples.
- Unify the examples.

C. Syntax Schemas

This section is non-normative.

This appendix contains XML schemas for validating RDF/XML forms. These are example schemas for information only and are not part of this specification.

C.1 RELAX NG Compact Schema

This section is non-normative.

This is an example schema in RELAX NG Compact (for ease of reading) for RDF/XML. Applications can also use the RELAX NG XML version. These formats are described in *Information technology -- Document Schema Definition Language (DSDL) -- Part 2: Regular-grammar-based validation -- RELAX NG* [*RELAXNG-SCHEMA*].

NOTE

The RNGC schema has been updated to attempt to match the grammar but this has not been checked or used to validate RDF/XML.

```
#
# RELAX NG Compact Schema for RDF/XML Syntax
#
# This schema is for information only and NON-NORMATIVE
#
# It is based on one originally written by James Clark in
# https://lists.w3.org/Archives/Public/www-rdf-comments/2001JulSep/024
# and updated with later changes.
#

namespace local = ""
namespace rdf = "http://www.w3.org/1999/02/22-rdf-syntax-ns#"
datatypes xsd = "http://www.w3.org/2001/XMLSchema-datatypes"
namespace its = "http://www.w3.org/2005/11/its"

start = doc

# I cannot seem to do this in RNGC so they are expanded in-line

# coreSyntaxTerms = rdf:RDF | rdf:ID | rdf:annotation | rdf:annotation
# syntaxTerms = coreSyntaxTerms | rdf:Description | rdf:li
# oldTerms    = rdf:aboutEach | rdf:aboutEachPrefix | rdf:bagID
# nodeElementIRIs       = * - ( coreSyntaxTerms | rdf:li | oldTerms )
# propertyElementURIs   = * - ( coreSyntaxTerms | rdf:Description | ol
# propertyAttributeIRIs = * - ( coreSyntaxTerms | rdf:Description | rd

# Also needed to allow rdf:li on all property element productions
# since we can't capture the rdf:li rewriting to rdf_<n> in relaxng

# Need to add these explicitly
xmllang = attribute xml:lang { text }
xmlbase = attribute xml:base { text }
# and to forbid every other xml:* attribute, element

doc =
```

```
  RDF | nodeElement

RDF =
  element rdf:RDF {
     xmllang?, xmlbase?, nodeElementList
}

nodeElementList =
  nodeElement*

  # Should be something like:
  #  ws* , (  nodeElement , ws* )*
  # but RELAXNG does this by default, ignoring whitespace separating t

nodeElement =
  element * - ( local:* | rdf:RDF | rdf:ID | rdf:about | rdf:parseType
                rdf:resource | rdf:nodeID | rdf:datatype | rdf:li |
                rdf:aboutEach | rdf:aboutEachPrefix | rdf:bagID ) {
      (idAttr | nodeIdAttr | aboutAttr )?, xmllang?, xmlbase?, propert
  }

  # It is not possible to say "and not things
  # beginning with _ in the rdf: namespace" in RELAX NG.

ws =
  " "

  # Not used in this RELAX NG schema; but should be any legal XML
  # whitespace defined by https://www.w3.org/TR/2000/REC-xml-20001006#


propertyEltList =
  propertyElt*

  # Should be something like:
  #  ws* , ( propertyElt , ws* )*
  # but RELAXNG does this by default, ignoring whitespace separating t

propertyElt =
  resourcePropertyElt |
  literalPropertyElt |
  parseTypeLiteralPropertyElt |
```

```
  parseTypeResourcePropertyElt |
  parseTypeTriplePropertyElt |
  parseTypeCollectionPropertyElt |
  parseTypeOtherPropertyElt |
  emptyPropertyElt

resourcePropertyElt =
  element * - ( local:* | rdf:RDF | rdf:ID |
                rdf:annotation | rdf:annotationNodeID |
                rdf:about | rdf:parseType |
                rdf:resource | rdf:nodeID | rdf:datatype | rdf:version
                its:dir | its:version |
                rdf:Description | rdf:aboutEach | rdf:aboutEachPrefix
                xml:* ) {
      idAttr?, (annotationAttr | annotationNodeIDAttr)?, xmllang?, xml
  }

literalPropertyElt =
  element * - ( local:* | rdf:RDF | rdf:ID |
                rdf:annotation | rdf:annotationNodeID |
                rdf:about | rdf:parseType |
                rdf:resource | rdf:nodeID | rdf:datatype | rdf:version
                its:dir | its:version |
                rdf:Description | rdf:aboutEach | rdf:aboutEachPrefix
                xml:* ) {
      idAttr? , (annotationAttr | annotationNodeIDAttr)?, datatypeAttr
  }

parseTypeLiteralPropertyElt =
  element * - ( local:* | rdf:RDF | rdf:ID |
                rdf:annotation | rdf:annotationNodeID |
                rdf:about | rdf:parseType |
                rdf:resource | rdf:nodeID | rdf:datatype | rdf:version
                its:dir | its:version |
                rdf:Description | rdf:aboutEach | rdf:aboutEachPrefix
                xml:* ) {
      idAttr?, (annotationAttr | annotationNodeIDAttr)?, parseLiteral,
  }

parseTypeResourcePropertyElt =
  element * - ( local:* | rdf:RDF | rdf:ID |
                rdf:annotation | rdf:annotationNodeID |
```

```
                rdf:about | rdf:parseType |
                rdf:resource | rdf:nodeID | rdf:datatype | rdf:version
                its:dir | its:version |
                rdf:Description | rdf:aboutEach | rdf:aboutEachPrefix
                xml:* ) {
      idAttr?, (annotationAttr | annotationNodeIDAttr)?, parseResource
  }

parseTypeTriplePropertyElt =
  element * - ( local:* | rdf:RDF | rdf:about | rdf:parseType |
                rdf:resource | rdf:nodeID | rdf:datatype | rdf:version
                its:dir | its:version |
                rdf:Description | rdf:aboutEach | rdf:aboutEachPrefix
                xml:* ) {
      idAttr?, (annotationAttr | annotationNodeIDAttr)?, parseResource
  }

parseTypeCollectionPropertyElt =
  element * - ( local:* | rdf:RDF | rdf:ID |
                rdf:annotation | rdf:annotationNodeID |
                rdf:about | rdf:parseType |
                rdf:resource | rdf:nodeID | rdf:datatype | rdf:version
                its:dir | its:version |
                rdf:Description | rdf:aboutEach | rdf:aboutEachPrefix
                xml:* ) {
      idAttr?, (annotationAttr | annotationNodeIDAttr)?, xmllang?, xml
  }

parseTypeOtherPropertyElt =
  element * - ( local:* | rdf:RDF | rdf:ID |
                rdf:annotation | rdf:annotationNodeID |
                rdf:about | rdf:parseType |
                rdf:resource | rdf:nodeID | rdf:datatype | rdf:version
                its:dir | its:version |
                rdf:Description | rdf:aboutEach | rdf:aboutEachPrefix
                xml:* ) {
      idAttr?, (annotationAttr | annotationNodeIDAttr)?, xmllang?, xml
  }

emptyPropertyElt =
   element * - ( local:* | rdf:RDF | rdf:ID |
                 rdf:annotation | rdf:annotationNodeID |
```

```
                rdf:about | rdf:parseType |
                rdf:resource | rdf:nodeID | rdf:datatype | rdf:versio
                its:dir | its:version |
                rdf:Description | rdf:aboutEach | rdf:aboutEachPrefix
                xml:* ) {
       idAttr?, (resourceAttr | nodeIdAttr | datatypeAttr )?, xmllang?
       xmlbase?, versionAttr?, dirAttr?, itsVersionAttr?, propertyAttr
   }

idAttr =
  attribute rdf:ID {
      IDsymbol
  }

nodeIdAttr =
  attribute rdf:nodeID {
      IDsymbol
  }

aboutAttr =
  attribute rdf:about {
      IRI
  }

propertyAttr =
  attribute * - ( local:* | rdf:RDF | rdf:ID |
                  rdf:annotation | rdf:annotationNodeID |
                  rdf:about | rdf:parseType |
                  rdf:resource | rdf:nodeID | rdf:datatype | rdf:versi
                  rdf:li | its:dir | | its:version
                  rdf:Description | rdf:aboutEach |
                  rdf:aboutEachPrefix | rdf:bagID |
                  xml:* ) {
      string
  }

resourceAttr =
  attribute rdf:resource {
      IRI
  }

datatypeAttr =
```

```
  attribute rdf:datatype {
      IRI
  }

parseLiteral =
  attribute rdf:parseType {
      "Literal"
  }

parseResource =
  attribute rdf:parseType {
      "Resource"
  }

parseTriple =
  attribute rdf:parseType {
      "Triple"
  }

parseCollection =
  attribute rdf:parseType {
      "Collection"
  }

parseOther =
  attribute rdf:parseType {
      text
  }

versionAttr =
  attribute rdf:version {
      text
  }

dirAttr =
  attribute its:dir {
      text
  }

itsVersionAttr =
  attribute its:version {
      text
```

```
  }

IRI =
  string

literal =
  any

IDsymbol =
  xsd:NMTOKEN

any =
  mixed { element * { attribute * { text }*, any }* }
```

§ D. Index

§ D.1 Terms defined by this specification

annotating §2.20

blank node elements §2.2

conforming RDF/XML document §3.

constraint-id §4.4

node elements §2.2

Property Attribute §2.5

property elements §2.2

RDF/XML Document §3.

top-level XML document element §6.2.1

§ D.2 Terms defined by reference

[*RDF12-CONCEPTS*] defines the following:

- 2.1 Version Labels
- base direction
- blank node identifier
- blank nodes
- concrete RDF syntaxes
- datatype IRI
- directional language-tagged strings
- IRIs
- language tag
- language-tagged strings
- lexical form
- literals
- nodes
- object type
- predicate
- RDF 1.2 Abstract Syntax
- RDF Document
- RDF graphs
- RDF namespace IRI
- RDF term
- RDF Vocabulary
- reifier
- relative IRI references
- resource
- string
- subject
- triple
- triple terms
- XML literal

[*RFC3870*] defines the following:

- Security Considerations Section

[*RFC3986*] defines the following:

- resolved

[*SVG10*] defines the following:

Section 21 Metadata

[*XML-INFOSET*] defines the following:

attribute information item

character information item

comment information item

Conformance

document information item

document type declaration information item

element information item

namespace information item

notation information item

processing instruction information item

unexpanded entity reference information item

unparsed entity information item

[*XML-NAMES*] defines the following:

local name

namespace name

NCName

prefix

QName

QNames

[*XML11*] defines the following:

2.12 Language Identification

3.1 Start-Tags, End-Tags, and Empty-Element Tags

Common Syntactic Constructs

content

Letter

Name

S

[*XMLSCHEMA11-2*] defines the following:

anyString

anyURI

int

white space normalization

[*XPATH*] defines the following:

Information Set Mapping

[*XPATH-DATAMODEL-30*] defines the following:

XPath sequence

[*XPATH-FUNCTIONS-30*] defines the following:

https://www.w3.org/TR/xpath-functions-30/#func-serialize

E. References

E.1 Normative references

[BCP47]

Tags for Identifying Languages. A. Phillips, Ed.; M. Davis, Ed. IETF. September 2009. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc5646

[NFC]

Web NFC API. Luc Yriarte; Samuel Ortiz; Don Coleman. W3C. 16 June 2015. W3C Working Group Note. URL: https://www.w3.org/TR/nfc/

[RDF-SYNTAX-GRAMMAR-19990222]

Resource Description Framework (RDF) Model and Syntax Specification. Ora Lassila. W3C. 22 February 1999. W3C Recommendation. URL: https://www.w3.org/TR/1999/REC-rdf-syntax-19990222/

[RDF12-CONCEPTS]

RDF 1.2 Concepts and Abstract Data Model. Gregg Kellogg; Olaf Hartig; Pierre-Antoine Champin; Andy Seaborne. W3C. 28 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-concepts/

[RDF12-N-TRIPLES]

RDF 1.2 N-Triples. Gregg Kellogg; Dominik Tomaszuk. W3C. 20 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-triples/

[RFC2119]

Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc2119

[RFC3870]

application/rdf+xml Media Type Registration. A. Swartz. IETF. September 2004. Informational. URL: https://www.rfc-editor.org/rfc/rfc3870

[RFC3986]

Uniform Resource Identifier (URI): Generic Syntax. T. Berners-Lee; R. Fielding; L. Masinter. IETF. January 2005. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3986

[RFC8174]

Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc8174

[SAX]

SAX – The Simple API for XML. Steve Cornish. October 1999. URL: https://accu.org/journals/overload/7/34/cornish_515/

[UNICODE]

The Unicode Standard. Unicode Consortium. URL: https://www.unicode.org/versions/latest/

[XML-INFOSET]

XML Information Set (Second Edition). John Cowan; Richard Tobin. W3C. 4 February 2004. W3C Recommendation. URL: https://www.w3.org/TR/xml-infoset/

[XML-NAMES]

Namespaces in XML 1.0 (Third Edition). Tim Bray; Dave Hollander; Andrew Layman; Richard Tobin; Henry Thompson et al. W3C. 8 December 2009. W3C Recommendation. URL: https://www.w3.org/TR/xml-names/

[XML11]

Extensible Markup Language (XML) 1.1 (Second Edition). Tim Bray; Jean Paoli; Michael Sperberg-McQueen; Eve Maler; François Yergeau; John Cowan et al. W3C. 16 August 2006. W3C Recommendation. URL: https://www.w3.org/TR/xml11/

[XMLBASE]

XML Base (Second Edition). Jonathan Marsh; Richard Tobin. W3C. 28 January 2009. W3C Recommendation. URL: https://www.w3.org/TR/xmlbase/

[XMLSCHEMA11-2]

W3C XML Schema Definition Language (XSD) 1.1 Part 2: Datatypes. David Peterson; Sandy Gao; Ashok Malhotra; Michael Sperberg-McQueen; Henry Thompson; Paul V.

Biron et al. W3C. 5 April 2012. W3C Recommendation. URL: https://www.w3.org/TR/xmlschema11-2/

[XPATH]

XML Path Language (XPath) Version 1.0. James Clark; Steven DeRose. W3C. 16 November 1999. W3C Recommendation. URL: https://www.w3.org/TR/xpath-10/

§ E.2 Informative references

[CHARMOD]

Character Model for the World Wide Web 1.0: Fundamentals. Martin Dürst; François Yergeau; Richard Ishida; Misha Wolf; Tex Texin et al. W3C. 15 February 2005. W3C Recommendation. URL: https://www.w3.org/TR/charmod/

[ITS]

Internationalization Tag Set (ITS) Version 1.0. Christian Lieske; Felix Sasaki. W3C. 3 April 2007. W3C Recommendation. URL: https://www.w3.org/TR/its/

[JSON-LD11]

JSON-LD 1.1. Gregg Kellogg; Pierre-Antoine Champin; Dave Longley. W3C. 16 July 2020. W3C Recommendation. URL: https://www.w3.org/TR/json-ld11/

[RDF-SCHEMA]

RDF Schema 1.1. Dan Brickley; Ramanathan Guha. W3C. 25 February 2014. W3C Recommendation. URL: https://www.w3.org/TR/rdf-schema/

[rdf-syntax-grammar]

RDF 1.1 XML Syntax. Fabien Gandon; Guus Schreiber. W3C. 25 February 2014. W3C Recommendation. URL: https://www.w3.org/TR/rdf-syntax-grammar/

[RDF12-INTEROP]

RDF 1.2 Interoperability. Pierre-Antoine Champin. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-interop/spec/

[RDF12-N-QUADS]

RDF 1.2 N-Quads. Gregg Kellogg; Dominik Tomaszuk. W3C. 22 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-quads/

[RDF12-NEW]

What's New in RDF 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-new/spec/

[RDF12-PRIMER]

RDF 1.2 Primer. Niklas Lindström; Pierre-Antoine Champin. W3C. 3 April 2025. DNOTE. URL: https://www.w3.org/TR/rdf12-primer/

[RDF12-SCHEMA]

RDF 1.2 Schema. Dominik Tomaszuk. W3C. 26 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-schema/

[RDF12-SEMANTICS]

RDF 1.2 Semantics. Peter Patel-Schneider; Dörthe Arndt; Enrico Franconi. W3C. 1 February 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-semantics/

[RDF12-TRIG]

RDF 1.2 TriG. Gregg Kellogg; Dominik Tomaszuk. W3C. 13 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-trig/

[RDF12-TURTLE]

RDF 1.2 Turtle. Gregg Kellogg; Dominik Tomaszuk. W3C. 29 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-turtle/

[RDFA-PRIMER]

RDFa 1.1 Primer - Third Edition. Ivan Herman; Ben Adida; Manu Sporny; Mark Birbeck. W3C. 17 March 2015. W3C Working Group Note. URL: https://www.w3.org/TR/rdfa-primer/

[RELAXNG-SCHEMA]

Information technology -- Document Schema Definition Language (DSDL) -- Part 2: Regular-grammar-based validation -- RELAX NG. ISO/IEC. 2008. URL: http://standards.iso.org/ittf/PubliclyAvailableStandards/c052348_ISO_IEC_19757-2_2008(E).zip

[SPARQL12-CONCEPTS]

SPARQL 1.2 Concepts. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-concepts/spec/

[SPARQL12-ENTAILMENT]

SPARQL 1.2 Entailment Regimes. Peter Patel-Schneider. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-entailment/

[SPARQL12-FEDERATED-QUERY]

SPARQL 1.2 Federated Query. Ruben Taelman; Gregory Williams. W3C. 26 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-federated-query/

[SPARQL12-GRAPH-STORE-PROTOCOL]

SPARQL 1.2 Graph Store Protocol. Andy Seaborne; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-graph-store-protocol/

[SPARQL12-NEW]

What's New in SPARQL 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-new/spec/

[SPARQL12-PROTOCOL]

SPARQL 1.2 Protocol. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-protocol/

[SPARQL12-QUERY]

SPARQL 1.2 Query Language. Olaf Hartig; Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 29 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-query/

[SPARQL12-RESULTS-CSV-TSV]

SPARQL 1.2 Query Results CSV and TSV Formats. Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-csv-tsv/

[SPARQL12-RESULTS-JSON]

SPARQL 1.2 Query Results JSON Format. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-json/

[SPARQL12-RESULTS-XML]

SPARQL 1.2 Query Results XML Format. Ruben Taelman; Dominik Tomaszuk; Thomas Pellissier Tanon. W3C. 27 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-xml/

[SPARQL12-SERVICE-DESCRIPTION]

SPARQL 1.2 Service Description. Ruben Taelman; Gregory Williams. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-service-description/

[SPARQL12-UPDATE]

SPARQL 1.2 Update. Ruben Taelman; Andy Seaborne; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-update/

[STRIPEDRDF]

RDF: Understanding the Striped RDF/XML Syntax. W3C. 2001. Team Submission. URL: http://www.w3.org/2001/10/stripes/

[SVG10]

Scalable Vector Graphics (SVG) 1.0 Specification. Jon Ferraiolo. W3C. 4 September 2001. W3C Recommendation. URL: https://www.w3.org/TR/SVG/

[SVG11]

Scalable Vector Graphics (SVG) 1.1 (Second Edition). Erik Dahlström; Patrick Dengler; Anthony Grasso; Chris Lilley; Cameron McCormack; Doug Schepers; Jonathan Watt; Jon Ferraiolo; Jun Fujisawa; Dean Jackson et al. W3C. 16 August 2011. W3C Recommendation. URL: https://www.w3.org/TR/SVG11/

[XPATH-DATAMODEL-30]

XQuery and XPath Data Model 3.0. Norman Walsh; Anders Berglund; John Snelson. W3C. 8 April 2014. W3C Recommendation. URL: https://www.w3.org/TR/xpath-datamodel-30/

[XPATH-FUNCTIONS-30]

XPath and XQuery Functions and Operators 3.0. Michael Kay. W3C. 8 April 2014. W3C Recommendation. URL: https://www.w3.org/TR/xpath-functions-30/

↥